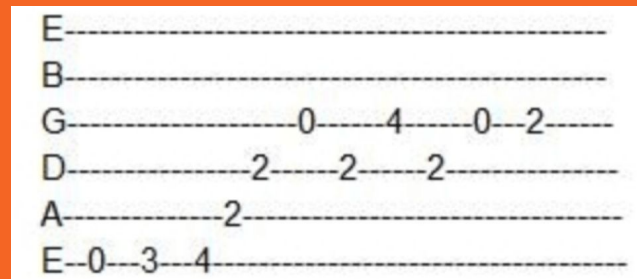

Computer Vision Transcription

Junyong, Vishaal, and Arnav



-
- **Retrieval of Guitarist Fingering Information Using Computer Vision (Joseph Scarr & Richard Green, 2010)**
 - **Guitar Tablature Generation Using Computer Vision (Brian Duke & Andrea Salgian, 2019)**
 - **A Computer Vision-Based Automatic Transcription of Guitar Music from RGBD Videos (Mark Asmar, 2022)**
 - **TapToTab : Video-Based Guitar Tabs Generation using AI and Audio Analysis (Ghaleb et al, 2024)**
-

Paper	Approach	Fretboard Detection	Fingertip Detection
Scarr & Green (2010)	Edge + skin color + contour filtering	Edge + Hough + Fret normalizing	Fingertips = top of filtered contour
Duke & Salgian (2019)	Skin color segmentation + contour + local maxima	PPHT + empirical fret spacing	Fingertips = local maxima on contour
Asmar (2022)	Mask R-CNN (fretboard) + MediaPipe Hands (21 landmarks) + depth	Mask R-CNN + Optical Flow	Fingertips = MediaPipe landmarks + depth check
Ghaleb et al. (2024)	YOLO + Fourier Transform	YOLO v8 OBB	YOLO (left-hand region overlap, not fingers)

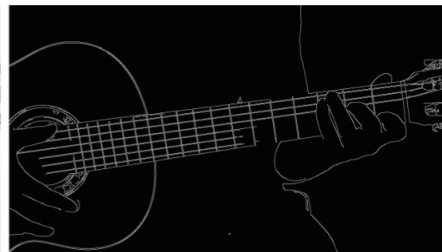
FretBoard Detection

Scarr & Green
(2010)

Duke & Salgian
(2019)



(a) Frame from a test video.



(b) Edge detection results.



(a) PPHT detected line segments.



(b) Detected strings and frets.

Scarr & Green
(2010)

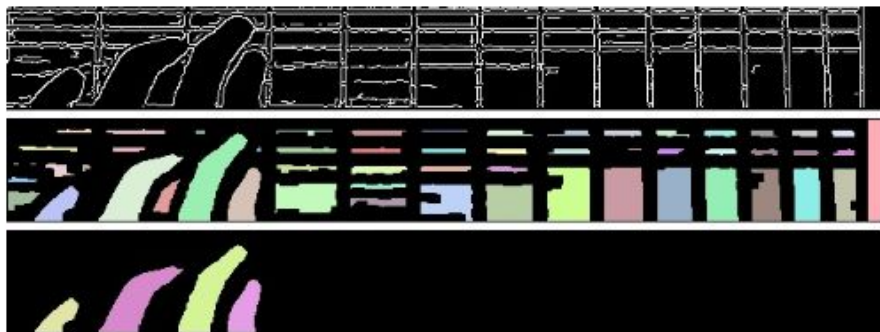


Figure 4: Finger detection. First Canny edge detection is performed and the result is dilated to create detectable contours. These contours are then filtered by the criteria in Equation 4 to identify the fingers.

Duke & Salgian (2019)



Fig. 5. Skin detection and segmentation.

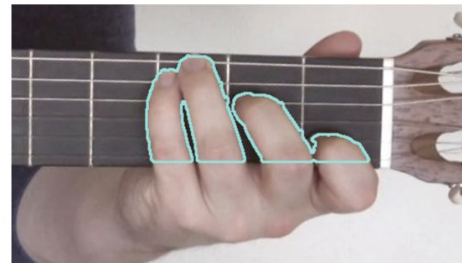


Fig. 6. Finger isolation and contour detection.

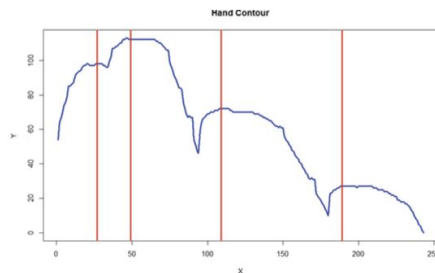


Fig. 7. Fingers contour with fingertips located at local maxima.

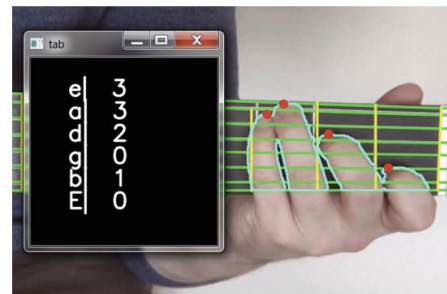


Fig. 8. System output. Tablature notation was flipped to correspond with strings in the video.

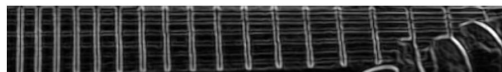
Asmar (2022)



(a) The RGB image used for the segmentation of the fretboard.



(b) The output of the segmentation model, a binary mask representing the pixels pertaining to the fretboard in white.

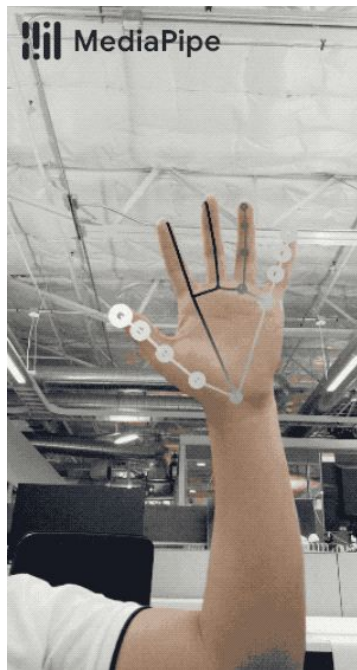


(a) The obtained image after applying both horizontal and vertical Sobel filters and adding their results.



(b) The resulting image after using adaptive thresholding.

Asmar (2022)



Step 1: Hand Pose Estimation

- Used **MediaPipe Hands** → gives **21 keypoints per hand** (fingertips, joints, wrist).
- Provides (x, y) and relative depth (z) for each joint.

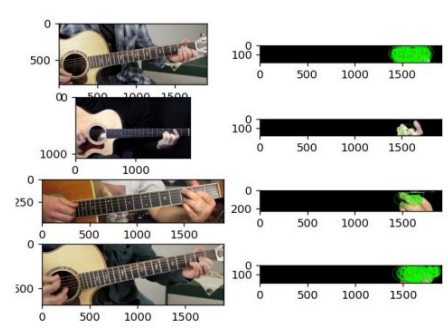
Step 2: Depth Analysis

- Compared fingertip z-coordinate with fretboard depth.
- If finger was **close enough to fretboard surface**, considered as pressing.

Step 3: Mapping to Strings & Frets

- Matched fingertip keypoints to nearest string and fret positions (extracted from segmentation + image processing).
-

Ghaleb et al.
(2024)



Model	Precision	Recall	MAP50	MAP50-95
YOLO V5	0.919	0.822	0.913	0.769
YOLO V8	0.987	0.995	0.992	0.951
YOLO V8 OBB	0.988	0.988	0.993	0.955
YOLO V9	0.958	0.949	0.983	0.907



Paper	Strengths	Weaknesses
Scarr & Green (2010)	Markerless pioneer, geometry-based, can infer frets	Low robustness, hover vs press missing, joint finger problem, sensitive to lighting
Duke & Salgian (2019)	Real-time, stable tracking, higher accuracy, simple setup	Edge-dependent, can't tell press vs hover, thin strings hard
Asmar (2022)	Deep learning robustness, depth-aware press detection, full pipeline (fret + fingers + pluck)	Heavy compute, dataset needed, depth errors
Ghaleb et al. (2024)	Higher accuracy, very fast	Needs audio + video

GuitarGuru (2023): Guitar Chord Detection

- The system accepts MP4 video files, YouTube links, or a live webcam feed. It then generates downloadable chord sheets, provides a real-time accuracy score for practice sessions, and stores user performance history.
 - The paper mentions a "finger pattern fretboard recognition system" but does not specify a method for detecting the fretboard itself. The analysis is solely based on the hand's geometry, independent of its position on the neck.
-

Approach	Fingertip Detection	Strengths	Weaknesses
Dataset: A custom dataset of 25,000 images (200×200 grayscale) was created, covering four chords (A, C, F, G) and a "No Chord" class. Architecture: A sequential CNN with 32 filters per layer, a (5,5) kernel size, and 'ReLU' activation function is used. Performance: The model achieved 97.09% accuracy on the test dataset.	Mediapipe Hands: Identifies 21 3D landmarks on the hand, providing a consistent skeletal representation of the fingers' positions, which serves as the input for chord classification.	High Accuracy: The model demonstrates a high classification accuracy of 97.09% on its specific dataset. Real-Time Feedback: Capable of analyzing live webcam feeds, providing immediate feedback for learners.	Extremely Limited Vocabulary: The system can only recognize four specific chords (A, C, F, G), making it impractical for most songs. No Voicing Distinction: Since the fretboard is not detected, the system cannot distinguish between different voicings of the same chord (e.g., an open position G vs. a barre G at the 3rd fret). Visual Condition Dependent: Performance is likely sensitive to lighting conditions, camera angles, and obstructions of the fretting hand.

Next Steps

- Further literature review
 - Implement some of the ideas in papers reviewed
 - Any thoughts?
-

Sep/29 Update

Computer Vision Transcription

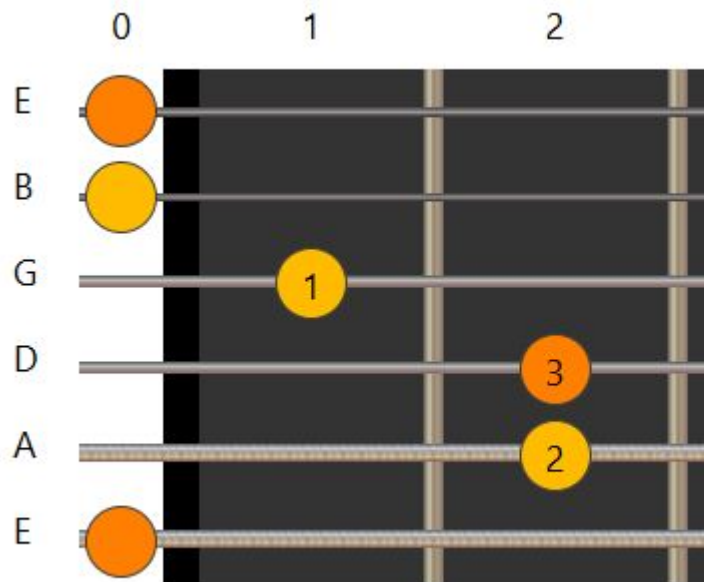
Junyong, Vishaal, and Arnav

MediaPipe

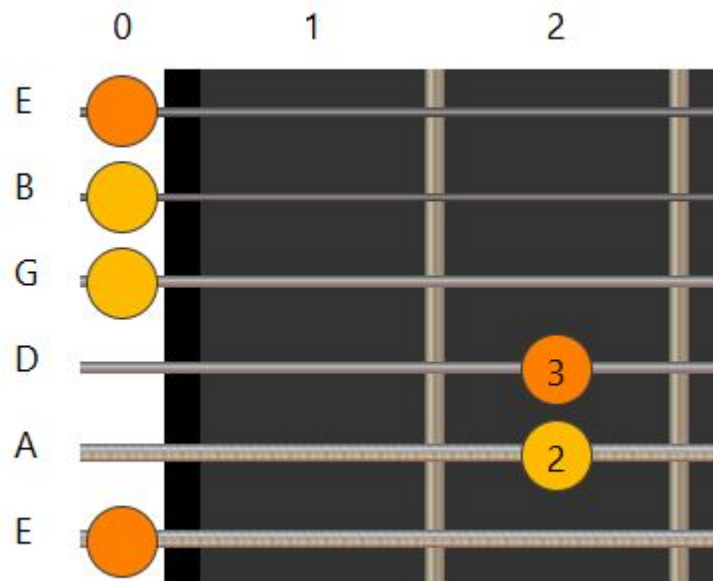


F3	B3	D#3	D3	C#3	C3	B2	A#2	A2	G#2	G2	F#2	F2	E2
A#4	A4	G#3	G3	F#3	F3	E3	D#3	D3	C#3	C3	B2	A#2	A2
D#4	D4	C#4	C4	B3	A#3	A3	G#3	G3	F#3	F3	E3	D#3	D3
G#4	G4	F#4	F4	E4	D#4	D4	C#4	C4	B3	A#3	A3	G#3	G3
C5	B4	A#4	A4	G#4	G4	F#4	F4	E4	D#4	D4	C#4	C4	B3
F5	E5	D#5	D5	C#5	C5	B4	A#4	A4	G#4	G4	F#4	F4	E4

E Major



E Minor



Finger overlap with strings means the string is being pressed down hard, light, or not at all.

This corresponds to the note being **played**, the note being **muted**, or the **open** string being played.

Guitar sees: [0, 2, 2, 1, 0, 0] (E major) but an E minor chord is being played.

List of possible chords: [0, 2, 2, 1, 0, 0], [0, 2, 2, 0, 0, 0], [0, 0, 2, 1, 0, 0], [0, 2, X, 1, 0, 0], etc.

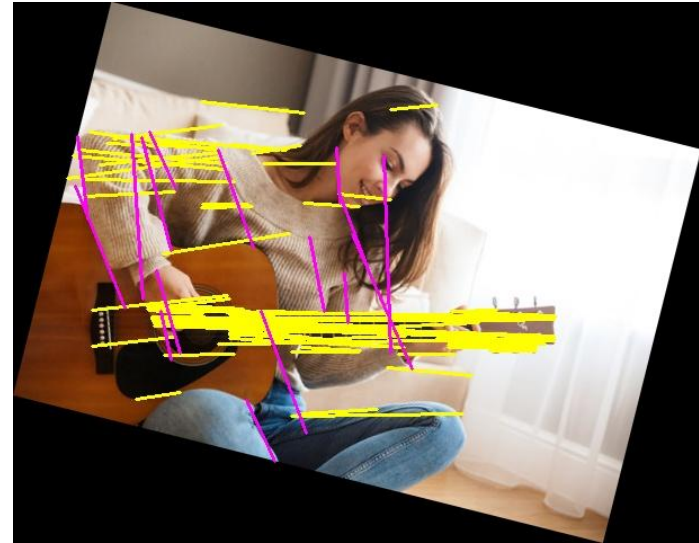
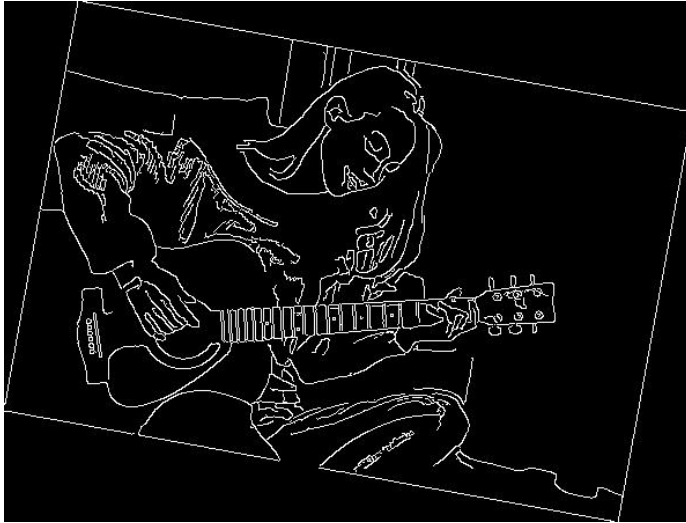
Results:

[0, 2, 2, 0, 0, 0]	->	1.1253
[0, 2, 2, 1, 0, 0]	->	1.1238
[0, 0, 2, 0, 0, 0]	->	1.0681
[0, 0, 2, 1, 0, 0]	->	1.0666
[0, 2, 0, 0, 0, 0]	->	1.0660

FretBoard Detection



FretBoard Detection



Next Steps

- Further literature review
 - Continue work on fretboard detection
 - Continue work on audio analysis
 - Start work on finger detection
 - Any thoughts?
-

Chord-detection:

- <https://github.com/AlbertMitjans/chord-detection>
- <https://github.com/Liannus/Chord-Calculator?tab=readme-ov-file#the-challenge>
- https://github.com/djbacad/guitar-chord-recognition?utm_source=chatgpt.com
- https://github.com/Liannus/Chord-Calculator?utm_source=chatgpt.com

Todo:

Dataset

Talk with person who did before

Fretboard detection

Finger detection

Good reference for MediaPipe hands:

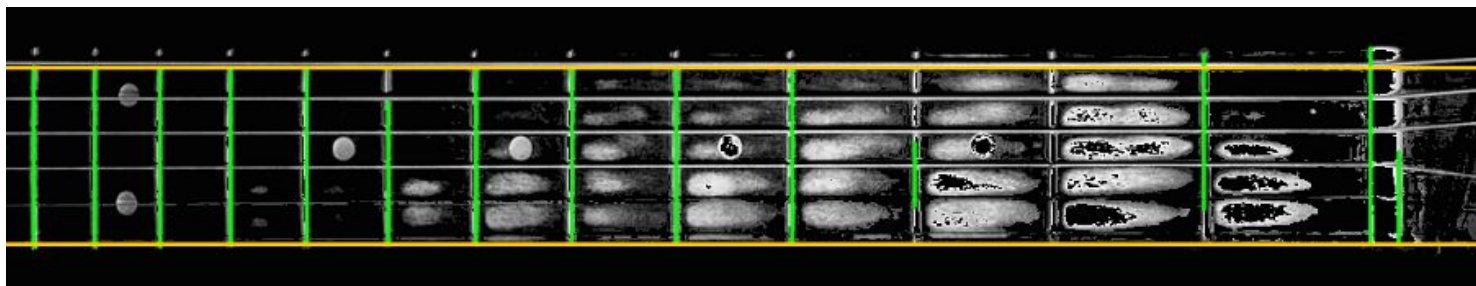
<https://github.com/akshaybahadur21/Guitar-Learner/blob/main/CreateDataset.py>

<https://github.com/akshaybahadur21/Guitar-Learner/blob/main/GuitarLearner.py>

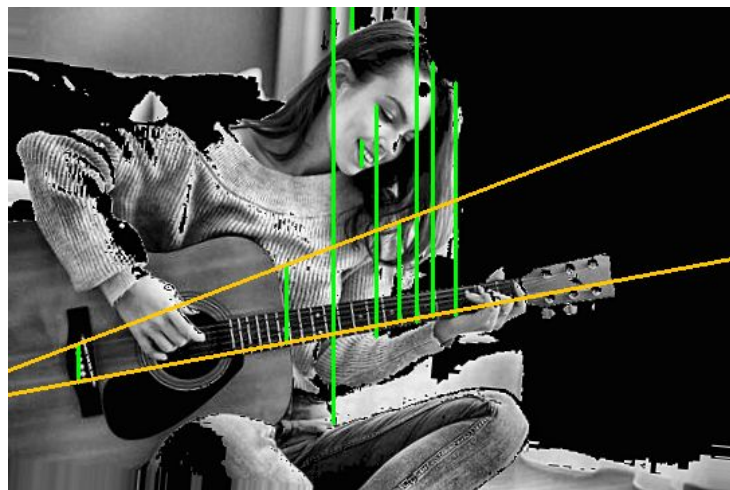
Oct/06 Update

Computer Vision Transcription

Junyong, Vishaal, and Arnav







Comprehensive Testing of FFT Analysis

Correctly identified **81%** of standard open chords and barre chords.

FAIL Cm expected=[-1, 3, 5, 5, 4, 3] got=[-1, 3, 3, 5, 1, 1]

FAIL D expected=[-1, -1, 0, 2, 3, 2] got=[-1, -1, 0, 1, 4, 4]

FAIL Dm expected=[-1, -1, 0, 2, 3, 1] got=[-1, -1, 0, 2, 3, 0]

FAIL Emaj@1 expected=[1, 3, 3, 2, 1, 1] got=[1, 1, 3, 2, 1, 1]

FAIL Amin@2 expected=[-1, 2, 4, 4, 3, 2] got=[-1, 2, 4, 4, 4, 0]

FAIL Amin@5 expected=[-1, 5, 7, 7, 6, 5] got=[-1, 3, 4, 6, 3, 2]

FAIL Amaj@7 expected=[-1, 7, 9, 9, 9, 7] got=[-1, 2, 2, 4, 4, 0]

FAIL Amaj@10 expected=[-1, 10, 12, 12, 12, 10] got=[-1, 6, 9, 7, 8, 5]

FAIL Amin@10 expected=[-1, 10, 12, 12, 11, 10] got=[-1, 6, 5, 7, 6, 3]

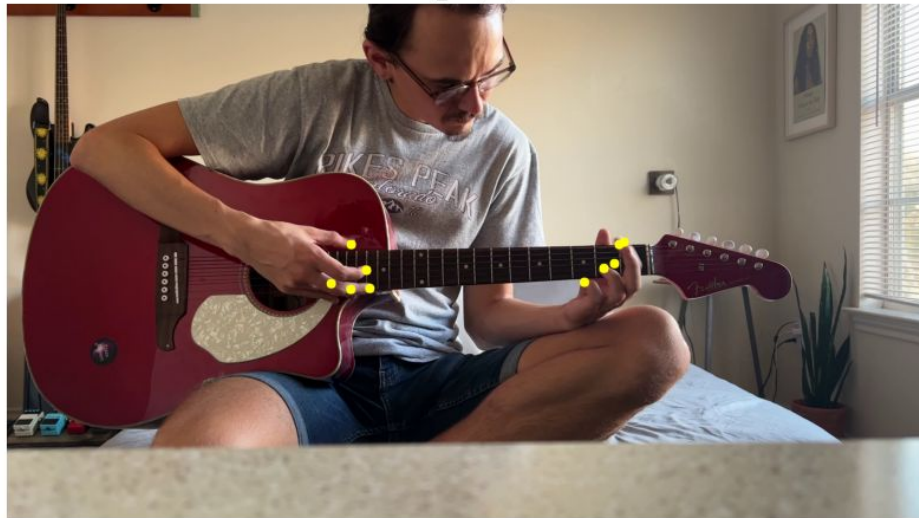
FAIL Amin@11 expected=[-1, 11, 13, 13, 12, 11] got=[2, 6, 6, 6, 6, 6]

Fret Detection



Fingertip Detection using Mediapipe

31_14.png



34_8.png



Datasets

- <https://www.kaggle.com/datasets/jacksonlightfoot/guitar-transcription-dataset>
- <https://universe.roboflow.com/ghaleb/guitar-fretboard>



Next Steps

- Investigate FFT analysis failures
 - Find more accurate measures of chord audio similarity
 - Further work on canny edge detection (crop fretboard somehow?)
 - Produce potential chords from MediaPipe fingers and canny edge overlap
 - Any thoughts?
-

Oct 20 Update

Computer Vision Transcription

Junyong, Vishaal, Arnav, and Basil

Fretboard Isolation

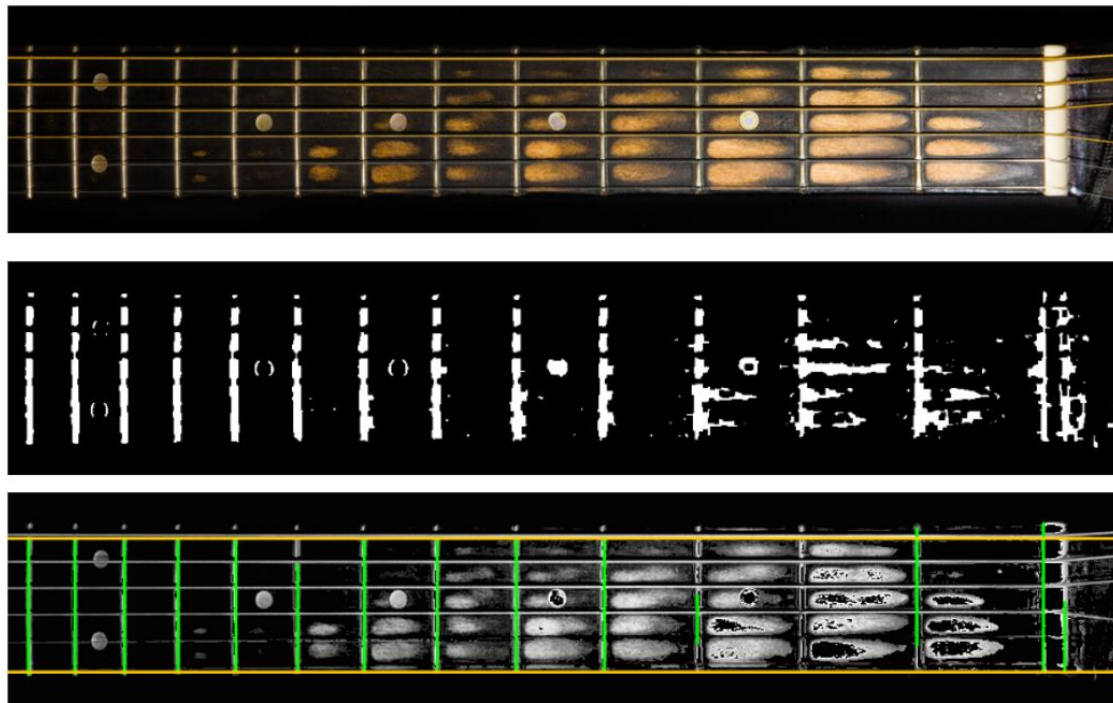
Prediction for 31_8.png



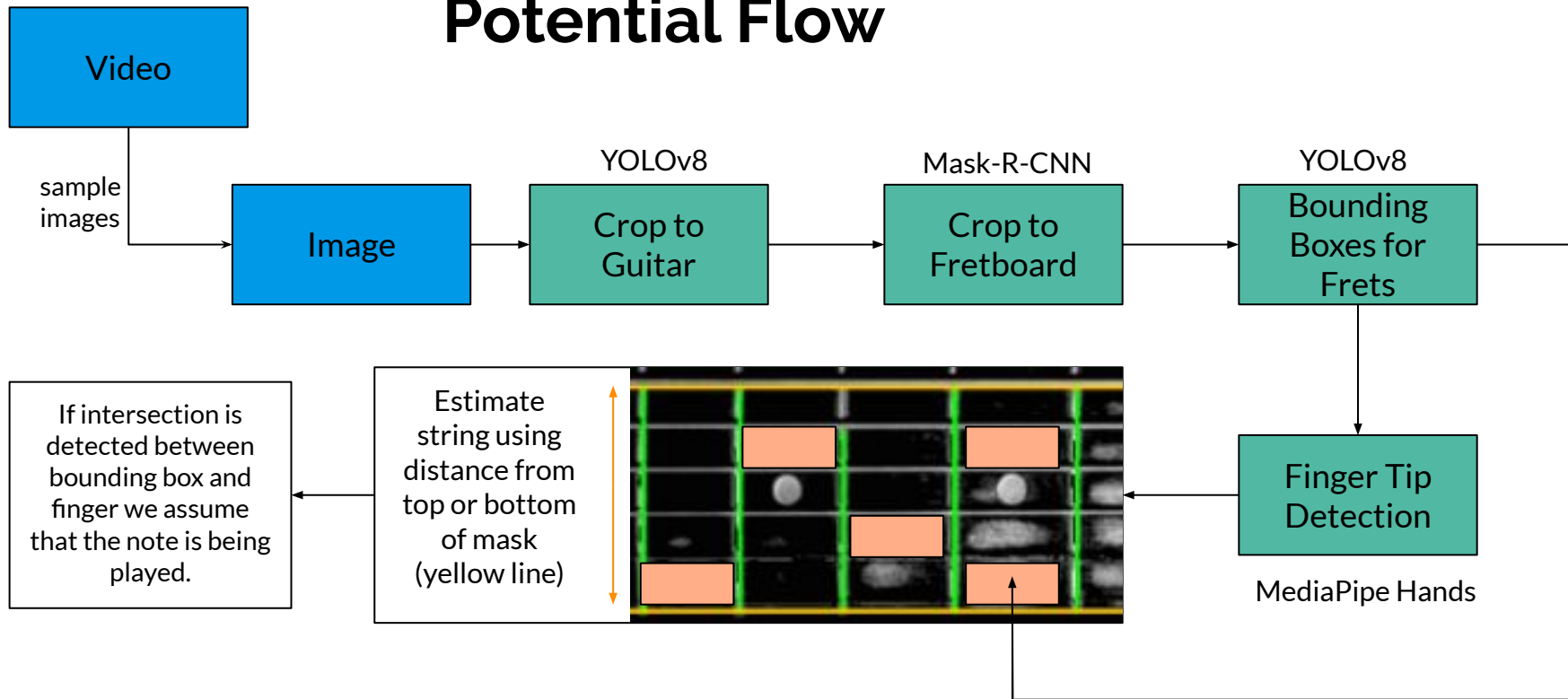
Prediction for 34_7.png



Swapping Canny Edge Detection for Estimation



Potential Flow



Next Steps

- Implement a basic version of our potential flow (skip some image processing steps)
 - Check fretboard segmentation model performance for low lighting images and make improvements like high contrast if required.
 - Evaluate performance and make changes from there
 - Any thoughts?
-

Oct 27 Update

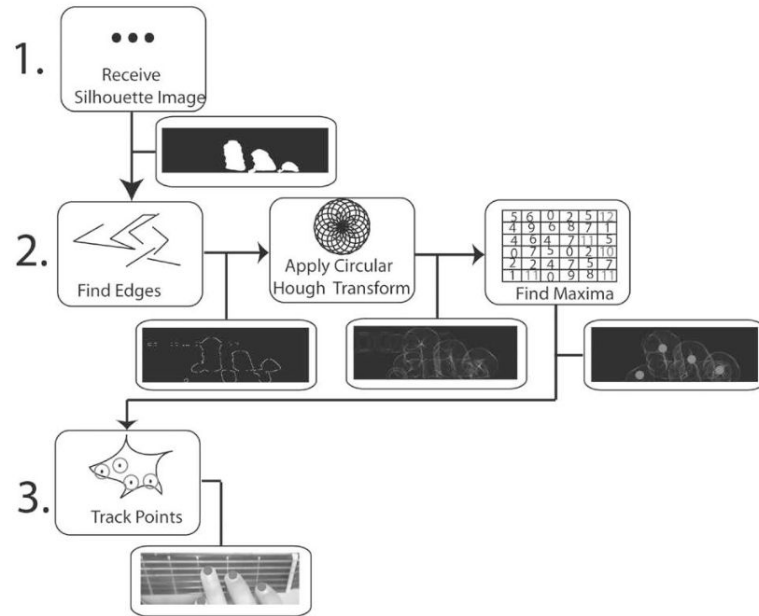
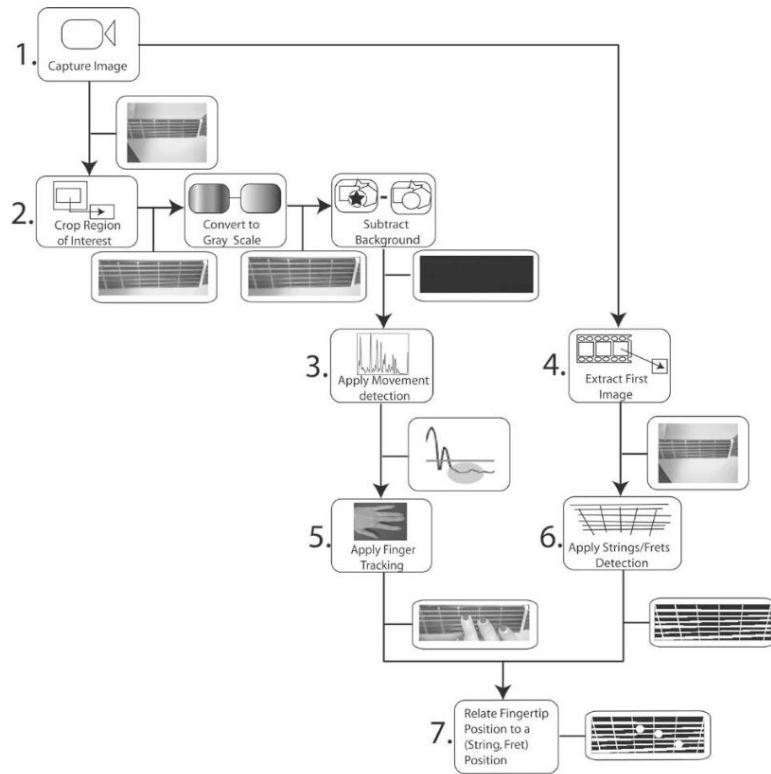
Computer Vision Transcription

Junyong, Vishaal, Arnav, and Basil

Literature Reviews

- *Computer Vision Method for Guitarist Fingering Retrieval* by Burns et al. (2011)
- Guitar Music Transcription from Silent Video (Goldstein & Moses, 2018)
- A 3D Guitar Fingering Assessing System Based on CNN-Hand Pose Estimation and SVR-Assessment (Zhao WANG and Jun OHYA, 2018)





A 3D Guitar Fingering Assessing System Based on CNN-Hand Pose Estimation and SVR-Assessment

- CNN Pose Estimation: A Convolutional Neural Network (CNN) is used to estimate the temporal 3D position of 16 hand joints from the video.
 - SVR Assessment: The 3D pose data is fed into a Support Vector Regression (SVR) model. This model is trained on scores from a human expert and learns to predict a 'score' for how well the fingering was performed
-

Next Steps

- Conduct further literature review
 - Main priority: finger/strings detection
 - Secondary: other components (frets, fretboard, etc)
 - Learn how to use Gilbreth (accessing, using Slurm, etc)
 - Any thoughts?
-

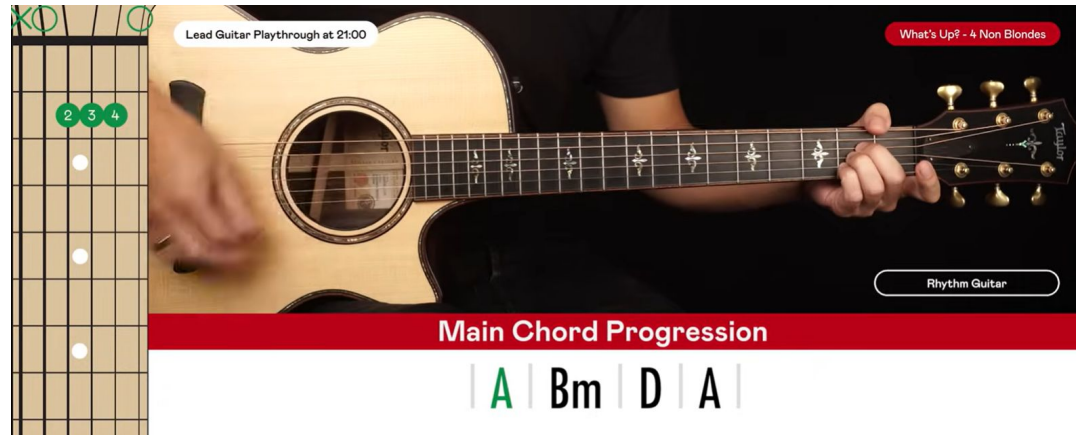
Nov 10 Update

Computer Vision Transcription

Junyong, Vishaal, Arnav, and Basil

Dataset Creation

- Scrape videos from online guitar tabs videos
- Two potential approaches:
 - Use labeled chords in videos to label stills of just the fretboard
 - Conduct chord audio analysis on fretboard video
- Use this dataset for model training



FFT Analysis

```
def _frame_audio(audio: np.ndarray, sr: int = SR, win_s: float = 0.046, hop_s: float = 0.023):
    win = int(win_s * sr)
    hop = int(hop_s * sr)
    if win < 32:
        win = 2048
    if hop < 16:
        hop = win // 2

    nfft = 1
    while nfft < win:
        nfft *= 2

    w = np.hanning(win).astype(np.float32)
    frames = []
    for start in range(0, max(1, len(audio) - win), hop):
        seg = audio[start:start + win]
        if len(seg) < win:
            seg = np.pad(seg, (0, win - len(seg)))
        X = np.fft.rfft(seg * w, n=nfft)
        frames.append(np.abs(X))

    if not frames:
        X = np.fft.rfft(np.pad(audio, (0, win - len(audio))), n=nfft)
        frames = [np.abs(X)]

    S = np.stack(frames, axis=0)
    freqs = np.fft.rfftfreq(nfft, d=1 / sr)
    spec = np.median(S, axis=0)
    spec = spec / (np.max(spec) + 1e-9)
    return spec, freqs

def _hz_to_idx(freqs: np.ndarray, hz: float) -> int:
    if hz <= freqs[0]:
        return 0
    if hz >= freqs[-1]:
        return len(freqs) - 1
    return int(np.argmax(np.abs(freqs - hz)))

def _candidate_harmonics_for_string(open_hz: float, fret: int, max_hz: float, max_harm: int = 6):
    f0 = freq_from_fret(open_hz, fret)
    hs = []
    for h in range(1, max_harm + 1):
        fh = f0 * h
        if fh > max_hz:
            break
        hs.append(fh)
    return hs

def _collect_expected_harmonics(frets, freqs: np.ndarray, max_harm: int = 6):
    expected = []
    for s_idx, f in enumerate(frets):
```

Dataset

<https://github.com/LincLabUCCS/Albert>

[Guitar-chord-classifier roboflow](#)

<https://universe.roboflow.com/hlcv-jtcas/guitar-chords-eu59l>

Next Steps

- Further literature review.
 - Learn how to use gilbreth more.
 - Reach out to guitar youtubers to see if we can get their data.
 - Reach out to paper authors to see if we can get their code.
-

Nov 17 Update

Computer Vision Transcription

Junyong, Vishaal, Arnav, and Basil

FFT Analysis

Template-Based Chord Recognition

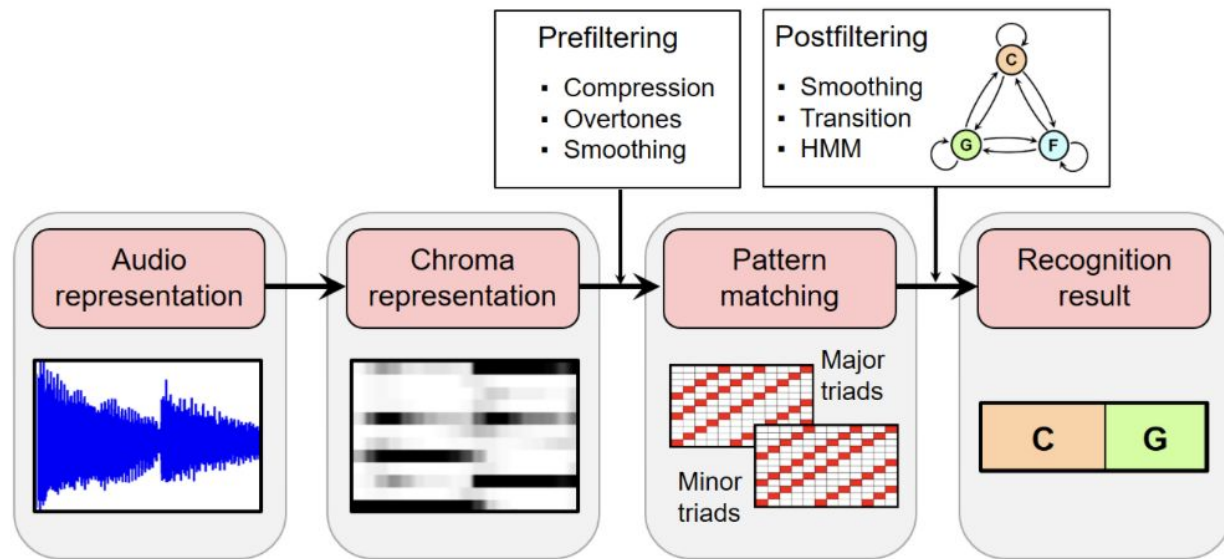


Figure 5.13 from [Müller, FMP, Springer 2015]

Dataset

<https://github.com/LincLabUCCS/Albert>

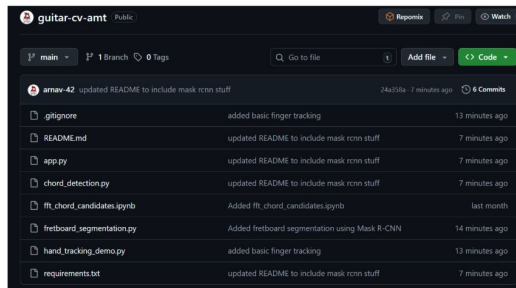
[Guitar-chord-classifier roboflow](#)

<https://universe.roboflow.com/hlcv-jtcas/guitar-chords-eu59l>

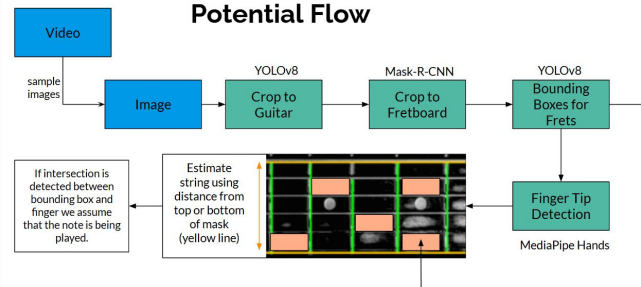
First draft of VIP Slides Complete

Demos

- `fft_chord_candidates.ipynb`: demo of detecting chords using a crude FFT analysis method
- `fretboard_segmentation.py`: demo of isolating fretboard using MaskRCNN
- `hand_tracking_demo.py`: demo of the finger tracking using MediaPipe Hands

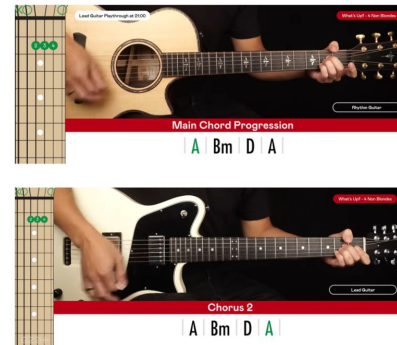


Potential Flow



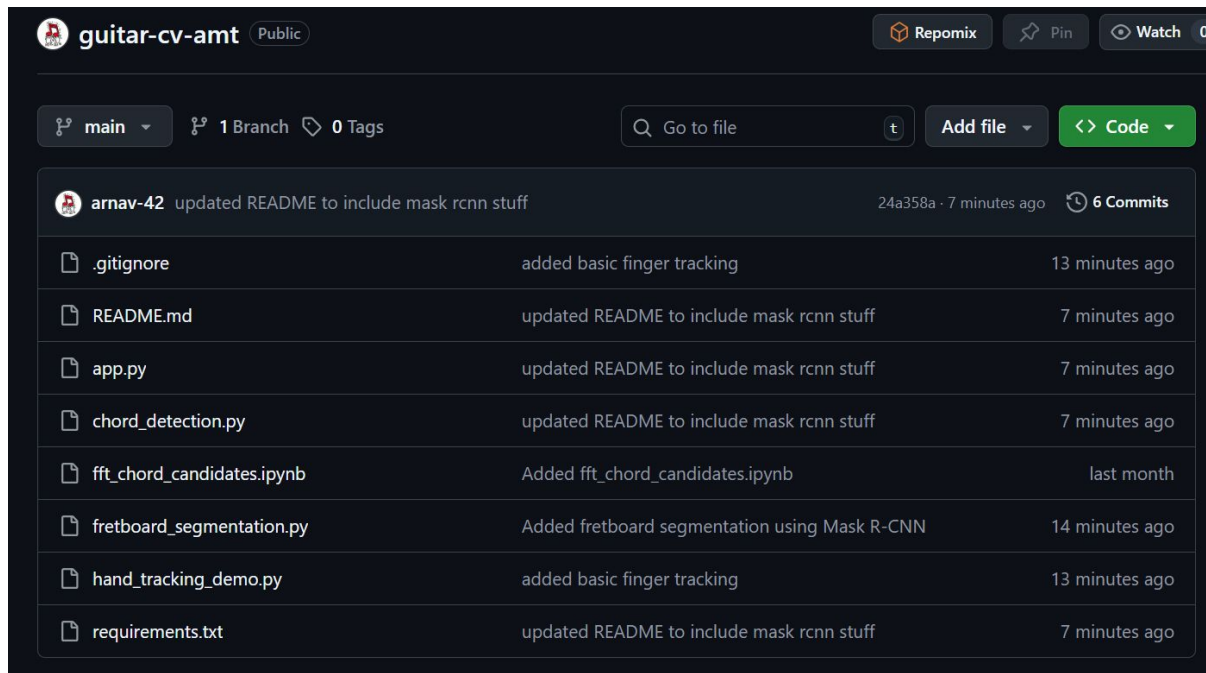
Dataset Creation

- Scrape videos from online guitar tabs videos
- Two potential approaches:
 - Use labeled chords in videos to label stills of just the fretboard
 - Conduct chord audio analysis on fretboard video
- Use this dataset for model training

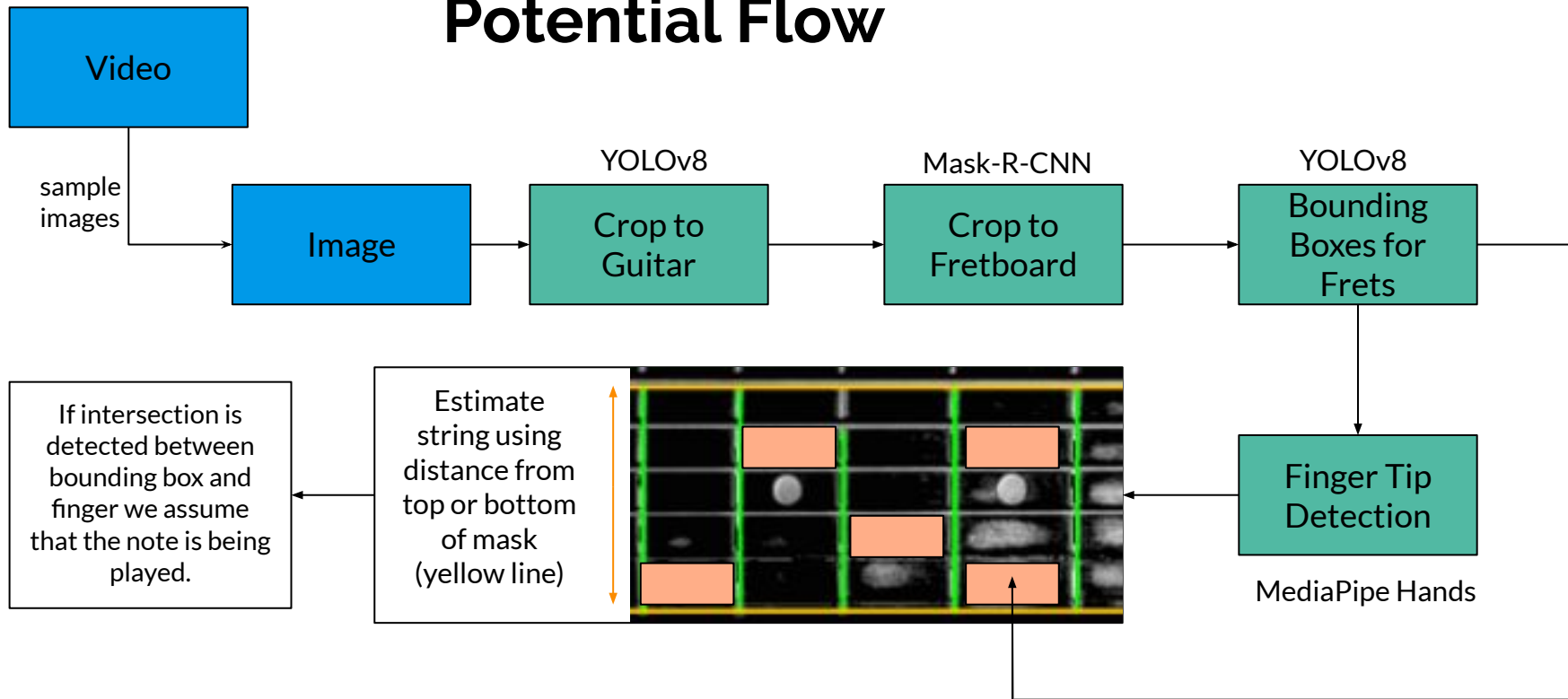


Demos

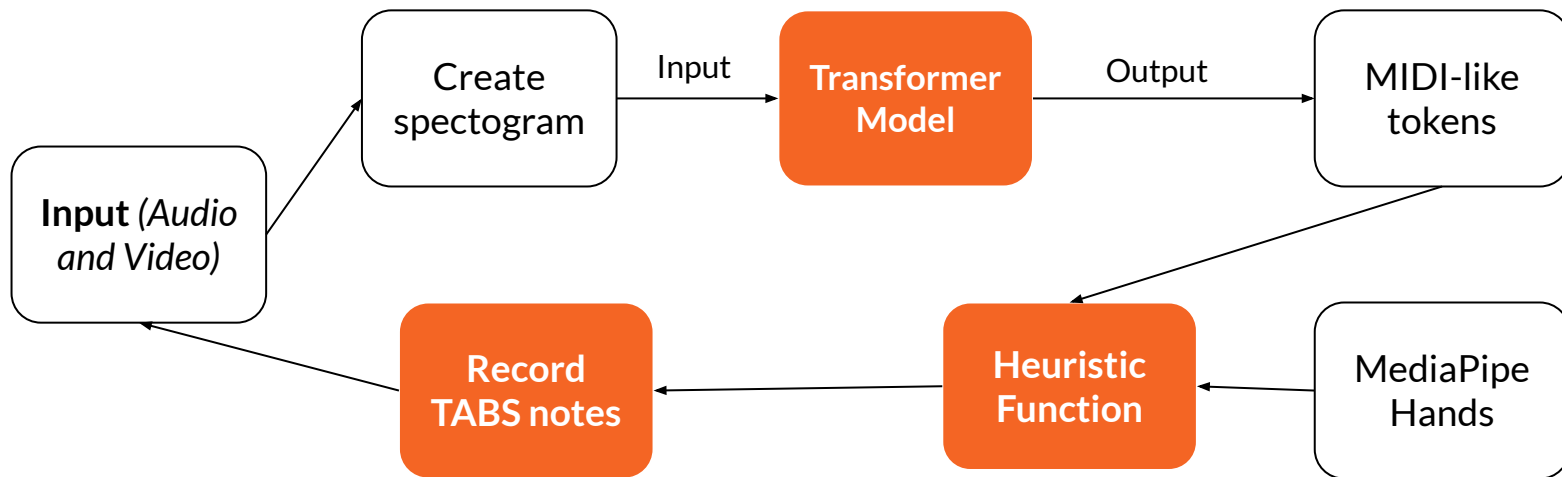
- **fft_chord_candidates.ipynb:** demo of detecting chords using a crude FFT analysis method
- **fretboard_segmentation.py:** demo of isolating fretboard using MaskRCNN
- **hand_tracking_demo.py:** demo of the finger tracking using MediaPipe Hands



Potential Flow



A Second Method (need feedback)

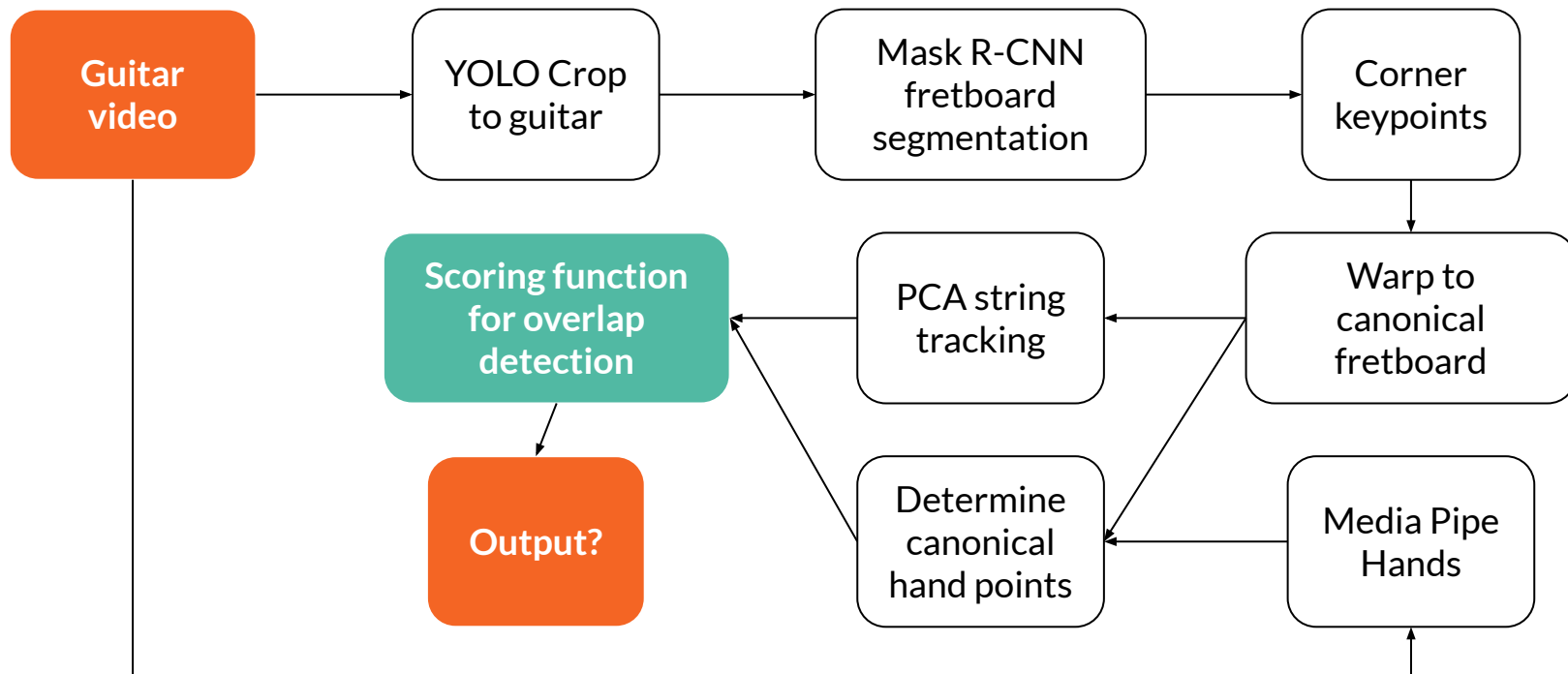


Jan 21 2026 Update

Computer Vision Transcription

Arnav and Vishaal

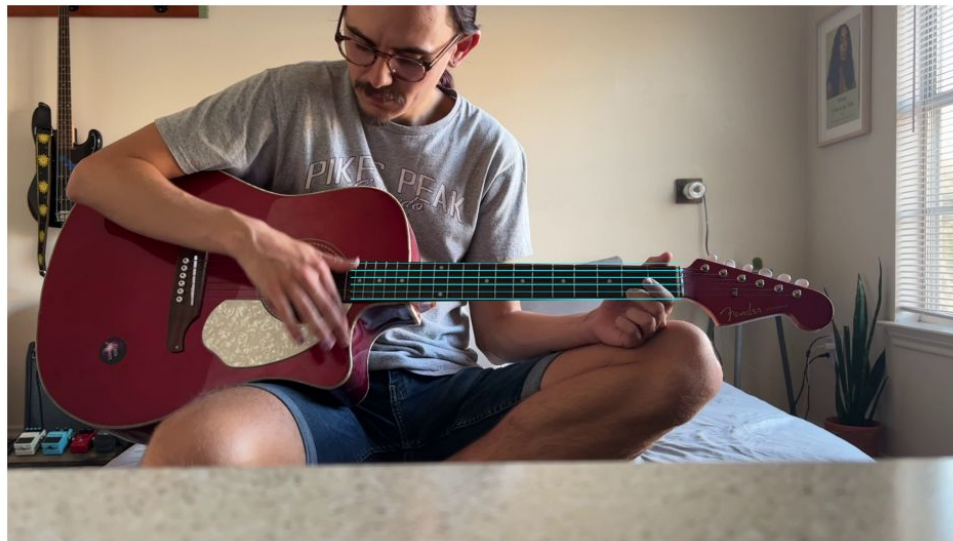
Canonicalization and fretboard isolation



String detection using PCA

- The binary segmentation mask is converted to coordinates.
 - Principal Component Analysis (PCA) is then applied to find the orientation of the guitar.
 - The pixel data is projected onto the boundaries of the fretboard.
 - Lines are drawn at 6 equidistant intervals to denote strings.
-

String detection using PCA



Updated fingertip detection

- The 'solutions' class in Mediapipe hands used for our fingertip detection became deprecated in January 12, 2026.
 - Now updated and fixed to use the Mediapipe Tasks API.
-

Next Steps

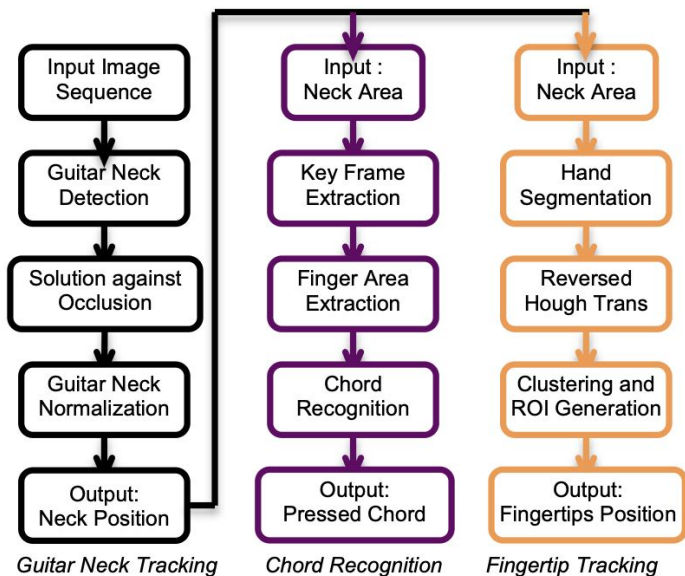
- Generalize to variable number of strings.
 - Update the rest of our codebase to fix deprecation issues
 - Further research on canonicalization of fretboard
-

Jan 28 2026 Update

Computer Vision Transcription

Arnav and Vishaal

Relooking at Duke & Salgian (2019)



- Achieved a 93.8% accuracy rate across 210 tested chords.
 - **Guitar Neck Tracking:** The neck area is projected to a standard resolution (1000 x 80 pixels) and the fret and string positions are calculated mathematically using the rule of 18.
 - **Chord Recognition:** The system detects “Key Frames” and maps fingertips to normalized neck positions to detect chord presses.
-

Fingertip tracking in Duke & Salgian

- A Bayesian Classifier separates skin-color pixels from the background to isolate the hand.
 - Reverse Hough transform then detects fingertips.
 - Clustering and ROI Generation: To remove noise, points detected over a buffer of 10 frames are grouped together.
-

Differentiating between pressing and hovering

- It is assumed that when a guitarist is transitioning between chords, their hand shape changes significantly to form the new chord.
 - If a finger is positioned over a specific fret/string coordinate during this critical window, it is considered a press.
 - The area must be at least 50% skin color to filter out background objects.
 - The finger contour must intersect with the lower edge of the guitar neck.
-

Onboarding new members

Provided new members with three papers we have based our work off of, each utilizing different approaches to:

- string detection
- fretboard/fret detection
- hand/finger tracking
- string played detection

After they've read them through (by next week?) they will start contributing to current work

Canonicalization of Guitar Fretboards from Video Frames

This notebook is a demo of extracting a canonical, top-down view of a guitar fretboard from raw video frames. This is an improvement on a previous demo that used only a Mask R-CNN seg mask. The original method worked fairly well, but struggled when the mask was noisy or partially occluded by hands.

Next steps: switch to YOLO?

```
import os
import numpy as np
import torch
import torchvision
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.models.detection.mask_rcnn import MaskRCNNPredictor
import torchvision.transforms as T
from PIL import Image
import cv2
import matplotlib.pyplot as plt

device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')
print(f"using {device}")

using cpu

def get_model_instance_segmentation(num_classes: int = 2):
    model = torchvision.models.detection.maskrcnn_resnet50_fpn(weights="DEFAULT")

    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

    in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
    hidden_layer = 256
    model.roi_heads.mask_predictor = MaskRCNNPredictor(
        in_features_mask,
        hidden_layer,
        num_classes,
    )
```

Results

- Canonical view works fairly well
- Still could be more accurate
 - replace mask R-CNN with YOLO
 - replace simple rectangle model with CNN keypoint model
 - Still need to train a proper keypoints model as well

Original Image



Cleaned Fretboard Mask



Warped Canonical View



Boring stuff



Running ...

arnav-42 chore: update demo site ✓ a7d7116 · 29 minutes ago 11 Commits		
docs	chore: update demo site	29 minutes ago
notebooks	refactor!: static site for demos, deprecated .py demos, intera...	39 minutes ago
src	refactor!: static site for demos, deprecated .py demos, intera...	39 minutes ago
.gitattributes	Added model_weight.pt for fretboard segmentation	2 months ago
.gitignore	added basic finger tracking	2 months ago
.jupyterlite.doit.db	refactor!: static site for demos, deprecated .py demos, intera...	39 minutes ago
README.md	chore: update README with link to demos	32 minutes ago
model_weights.pt	Added model_weight.pt for fretboard segmentation	2 months ago
requirements.txt	updated README to include mask rcnn stuff	2 months ago

Next steps

- Change to using YOLO for fretboard segmentation
 - Look into using CNNs for keypoint classification
 - Look into implementing some of the methods from Duke & Salgian (especially fretboard segmentation and strings)
 - Classifying overlap between MediaPipe Hands and strings (FFT code from last semester might come in handy here)
-

Feb 4 2026 Update

Computer Vision Transcription

Arnav, Vishaal, Om, Sooji

Onboarding of New Members

Onboarding - Sooji

1. Guitar Tablature Generation Using Computer Vision (2019, Brian Duke & Andrea Salgian)

Problem: Audio AMT cannot determine guitar fingering.

Main Idea: Generate guitar tablature using only a webcam not using inconvenient equipments for players

Key point:

Accuracy jump 52% to 86% comes from **switching from finger detection to fingertip contour maxima**.

Limitation:

- Strong dependence on recording conditions (background, lighting, camera angle)
- Skin color rule (Kovac model) is fragile across players
- Cannot distinguish hidden or hovering fingers
- Processes every frame -> jitter & inefficiency

Onboarding - Sooji

2. TapToTab : Video-Based Guitar Tabs Generation using AI and Audio Analysis (2024, Ghaleb, Ali, et al)

Problem: Existing guitar tab systems use only audio or only vision.

Main Idea: Synchronize **YOLO (vision)** with **Fourier (audio)** to infer exact string & fret.

Key point:

- YOLOv8 OBB for rotated fretboard detection
- Fourier-based frequency mapping
- Onset detection for A/V sync
- Zone-based fret classification

Limitation:

- Audio and video timing can get out of sync
- Zones don't give exact fret positions, still have ambiguity
- Fingers can be hidden from the camera

2/4 Progress Update - Om

Guitar Tablature Generation Using Computer Vision (Duke & Salgian 2019):

- Potentially valuable:
 - Utilized OpenCV to detect steel strings on a guitar (Canny edge detector)
 - Combine edge detection and prior knowledge of string/fretboard ratios
- Learnable Outcomes:
 - Fingertip detection using RGB values (potential issues)
 - Fingertip detection might have issues determining if a finger is resting vs playing a note
- Interesting Notes:
 - Open notes weren't considered
 - Skin pixel detection also picked up the guitar body
 - Only the absolute tip of the finger is taken into account for played notes

2/4 Progress Update - Om

A Computer Vision-Based Automatic Transcription of Guitar Music from RGBD Videos (Asmar 2022):

- Potentially valuable:
 - MediaPipe used for hand pose estimation, focused on the whole finger
 - Picking detection using optical flow for motion detection
- Learnable Outcomes:
 - Mask R-CNN used, similar to current approach
 - Both hands were detected but only one is present in the crop
- Interesting Notes:
 - Separate “picking” vs “non-picked” string calculations
 - Final average string played accuracy: 57.54%
 - Utilizes depth as an extra source of information

—

Current Work/Progress

Using rule of 18 with canonicalization



Segmentation Mask



Using YOLO over Mask R-CNN

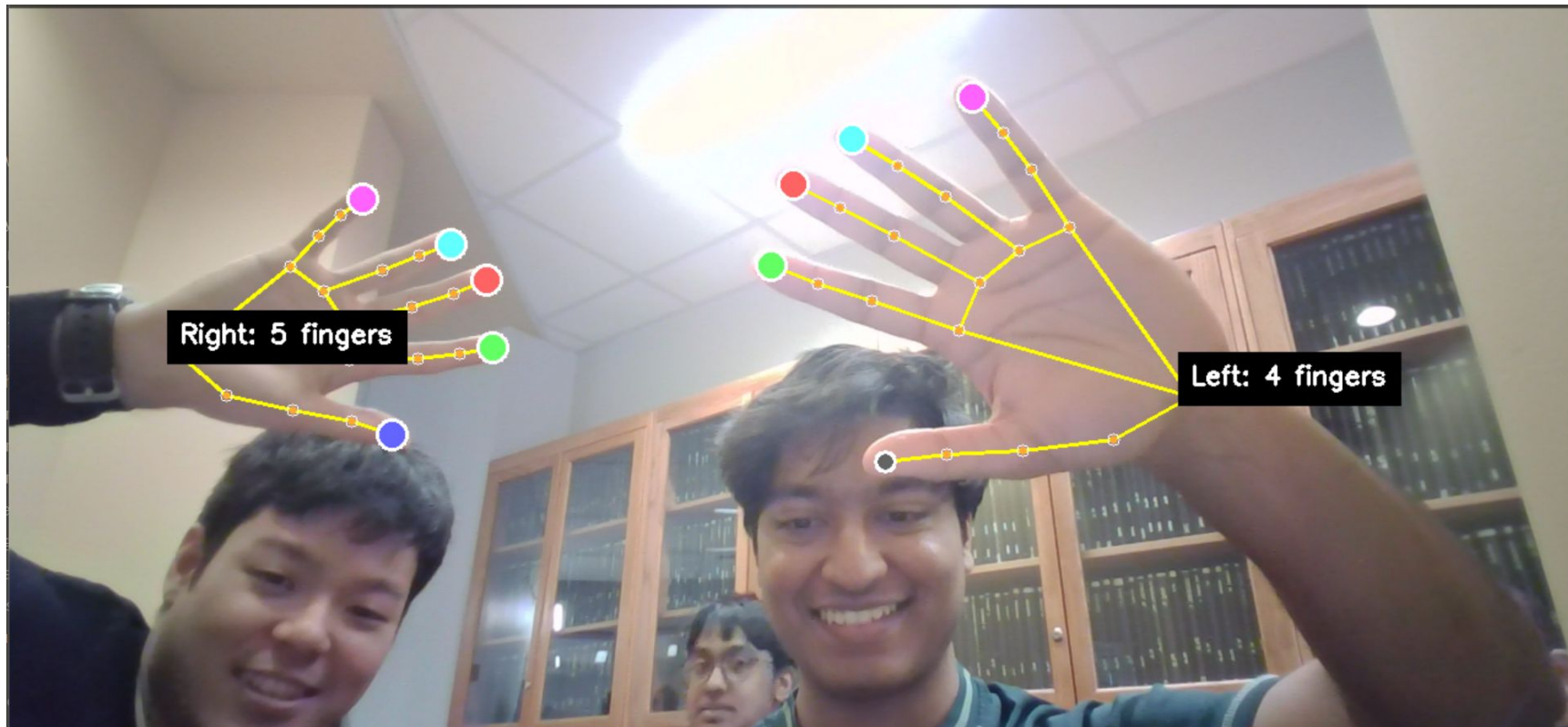
There are 2 approaches to detecting frets:

1. Training a model to detect frets from scratch. (We have the dataset used in Tap2Tab for this, but limited data since we need labeled frets.)
2. Using the rule of 18 with a rectangular mask to avoid warping the image so the rule of 18 applies.

To apply a rectangular mask: use YOLO which generates rectangular masks or use some other object detection model.

Tap2Tab uses YOLO to detect frets from scratch as well.

MediaPipe Hands Fix

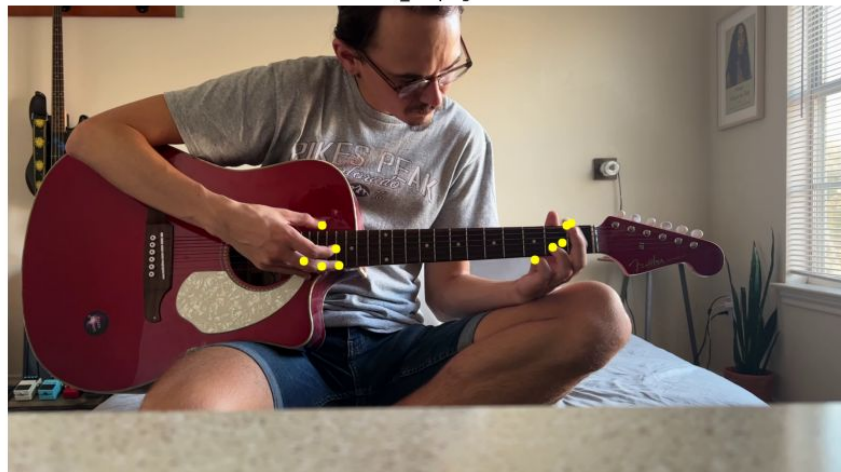


Combining MediaPipe Hands and fretboard

Prediction for 31_8.png



31_14.png



Next Steps

- Use YOLO to implement both fret detection approaches.
 - Work on mapping fingertips to strings now that we have Mediapipe for fingertip tracking and the mask-based string detection method using PCA.
-

Feb 11 2026 Update

Computer Vision Transcription

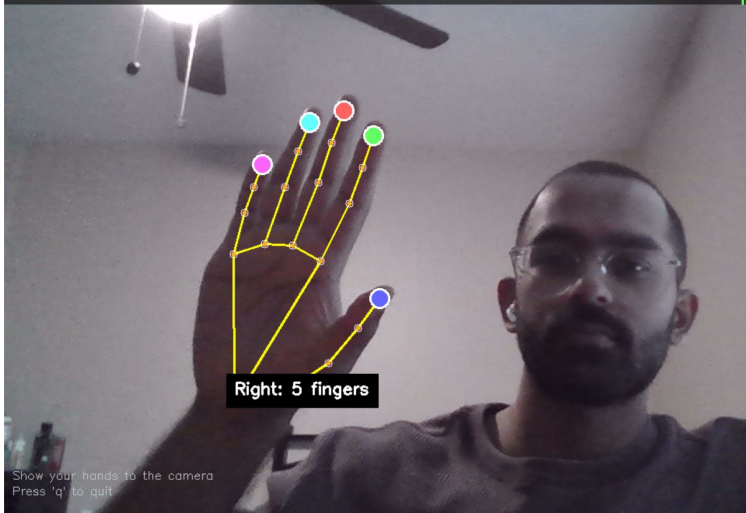
Arnav, Vishaal, Om, Sooji

2/11 Progress Update - Om

- Familiarized myself with the repository
 - Loaded and ran the demos locally
 - Created a bare bones prototype for finger depth
-

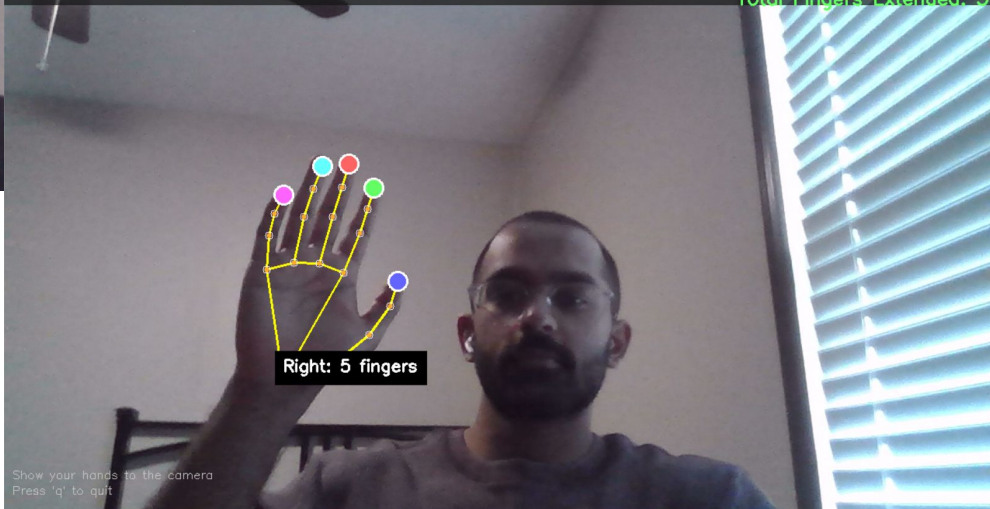
Finger Tracking Demo for Guitar AMT

Closest Finger: Index
Total Fingers Extended: 5



Finger Tracking Demo for Guitar AMT

Closest Finger: Pinky
Total Fingers Extended: 5



2/11 progress Sooji

- Finish reading Mark Asmar's paper (using RGBD video)
 - CV-based guitar tab transcription system
 - Pipeline: Mask R-CNN → MediaPipe → Optical flow
 - Depth solves hand occlusion issues
 - Ran demo codes from GitHub
 - Tested Mask R-CNN segmentation pipeline
 - Homography transformation for canonical view
-

Paleari & Huet

- Multimodal approach that uses both audio and video.
 - Fretboard detection is video only though.
 - Frets are identified by finding peaks in the projection of the original image and then mapped back to the original image by using the inverse of the rotation and skew transformations.
-

Projecting the image

- Used a Mask R-CNN to isolate the guitar, then applied a Hough transform to edge data (from Canny detection) to calculate the neck's dominant angle.
 - The image is then rotated to make the neck horizontal, cropped to the fretboard's height using horizontal projection, and then sheared iteratively until the variance of the vertical projection is maximized, ensuring the frets are perfectly vertical.
-

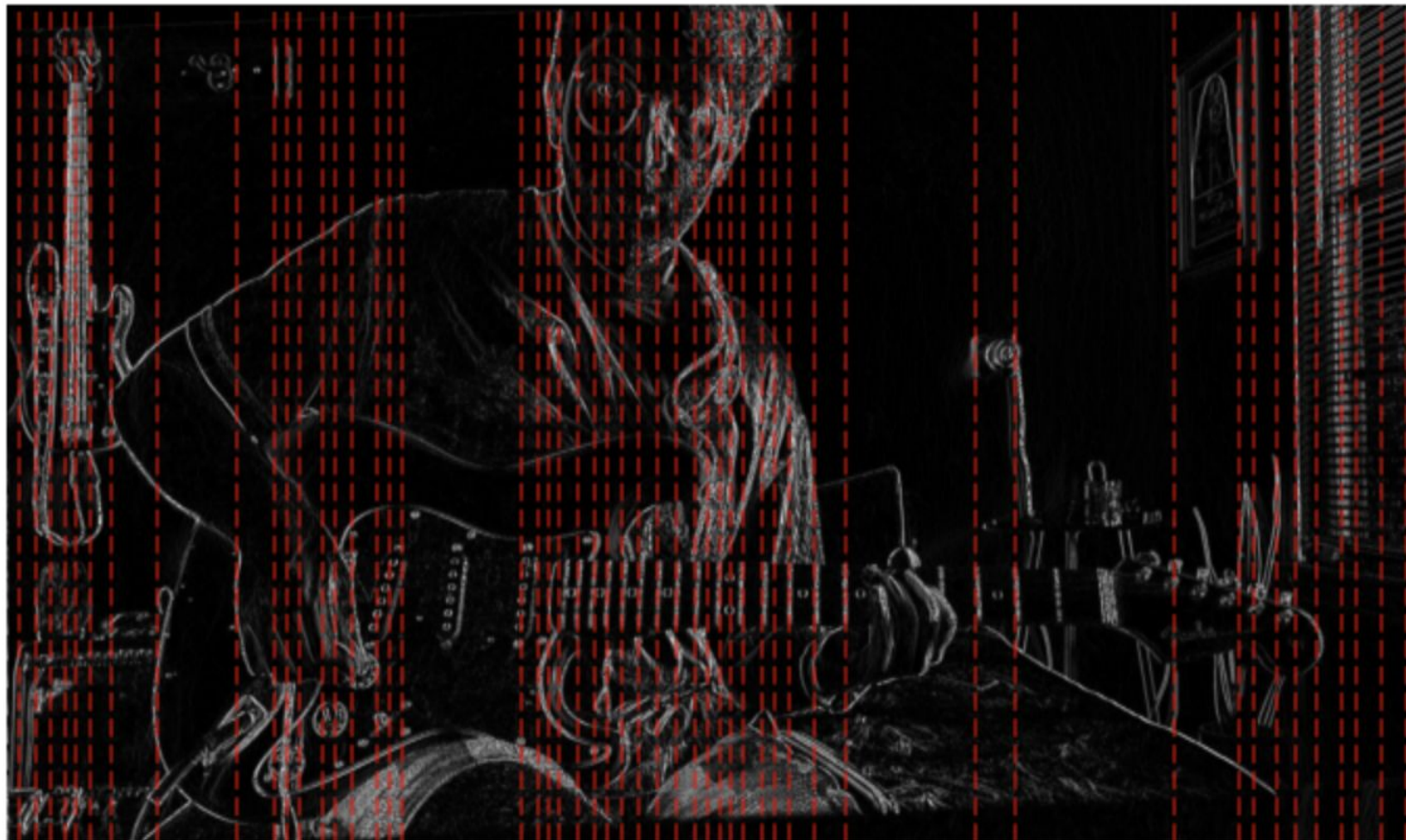
1. Input & Segmentation (Deep Learning)



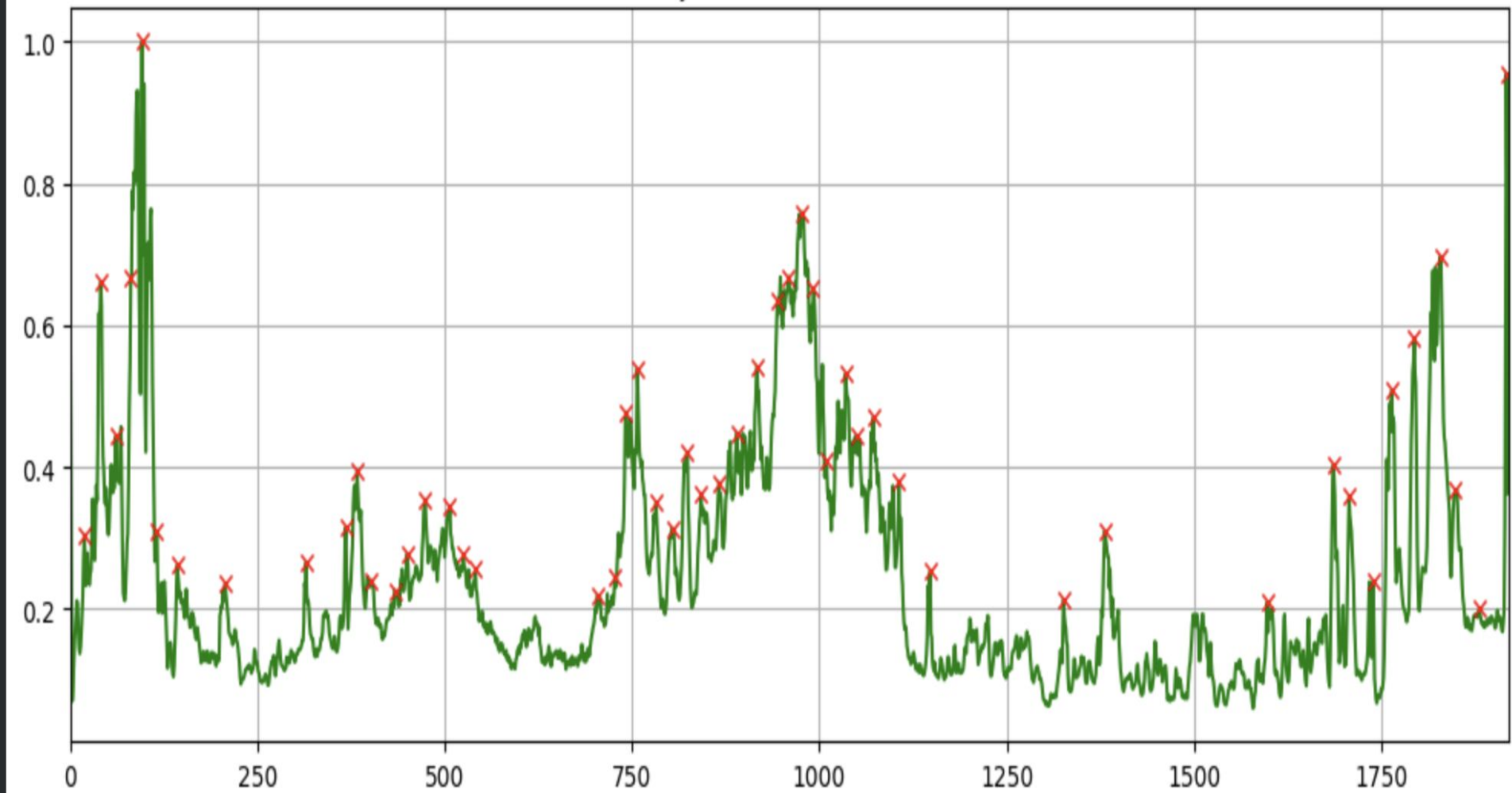
2. Rotated (1.8°) & Enhanced (Sobel/Wavelet Proxy)



3. Skewed (Factor -0.03) & Detected Frets



4. Vertical Projection Peaks (Fret Candidates)



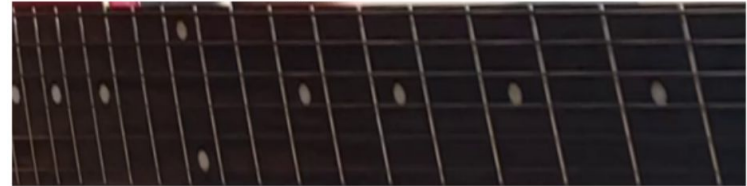


Corner points for canonicalization

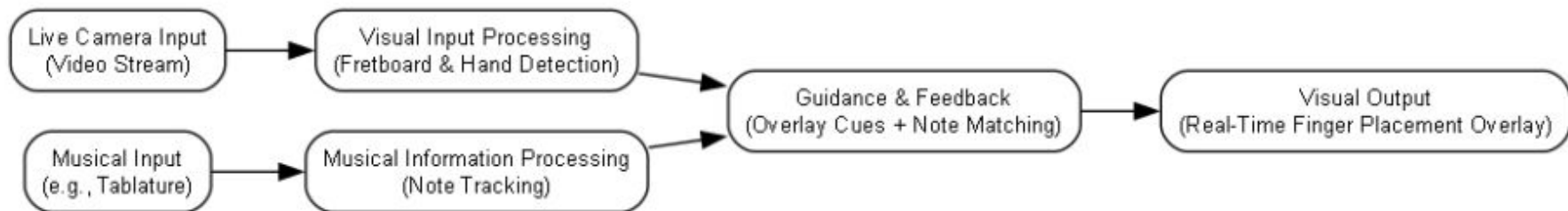
Original Image (contour + corners)



Warped Canonical View



Lit Review: Kulikauskaite 2025



Lit Review: Kulikauskaite 2025

Approach	Use Case	Implemented?
MediaPipe Hands	Fretted fingers tracking	Yes
MediaPipe Hands	Strum/pluck tracking	Yes
Canny Edge Detection	Frame preprocessing	Yes
Hough Line Transform	String, fret detection	In progress
“Rule of 18”	Fret detection	Yes
Template Matching	Fretboard isolation	No

Applications of Kulikauskaite 2025



Applications of Kulikauskaite 2025



Next Steps

- Finish Rule of 18 approach to fret detection
 - Relook at Hough Transforms for fret detection
 - Look at combining both approaches (re: Kulikauskaite)
 - Evaluate fret detection on Tap2Tab dataset
-
- Look at template matching for fretboard isolation
 - Move forwards past canonicalization, can revisit later
-
- Tracking fingertip + string overlap
-

Feb 18 2026 Update

Computer Vision Transcription

Arnav, Vishaal, Om, Sooji

Evaluating fret detection

Dataset	Precision	Recall	F1-Score	TP	FP	FN
valid	0.0951	0.0339	0.0500	35	333	997
test	0.1022	0.0406	0.0581	19	167	449
=====						

- Since frets are thin structural lines rather than standard objects, we evaluate performance using a Geometric Matching approach.
 - We mathematically calculate the perpendicular distance from every Ground Truth point to every line using the Hungarian Algorithm.
 - A prediction is counted as True Positive (TP) if it lands within 15 pixels of the ground truth.
-

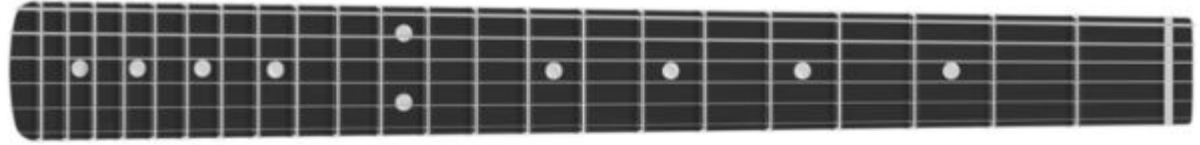
Evaluating fret detection

Dataset	Precision	Recall	F1-Score	TP	FP	FN
valid	0.0951	0.0339	0.0500	35	333	997
test	0.1022	0.0406	0.0581	19	167	449

=====

- Precision: Of the lines drawn, how many were actual frets?
 - Recall: Of the frets that exist, how many did we find?
 - High False Positives implies the Horizontal Projection step is failing to crop the image effectively. Vertical lines in the background are being labeled as frets.
 - The low Recall confirms that the crop is missing the neck of the image.
 - **Conclusion: Must improve masking and cropping steps since they are too loose. Must enforce stricter masking before detecting lines.**
-

Template Matching for Fretboard



Input



Template Matching for Fretboard

```
def preprocess_for_matching(img_bgr, blur_ksize=5):  
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)  
    if blur_ksize and blur_ksize > 1:  
        gray = cv2.GaussianBlur(gray, (blur_ksize, blur_ksize), 0)  
    edges = cv2.Canny(gray, 50, 150)  
    edges = cv2.dilate(edges, np.ones((3,3), np.uint8), iterations=1)  
    return edges  
  
temp_proc = preprocess_for_matching(template)  
inp_proc = preprocess_for_matching(inp)  
  
plt.figure(figsize=(10,4))  
plt.subplot(1,2,1); plt.imshow(temp_proc, cmap="gray"); plt.axis("off"); plt.title("Template edges")  
plt.subplot(1,2,2); plt.imshow(inp_proc, cmap="gray"); plt.axis("off"); plt.title("Input edges")  
plt.show()
```

Template edges

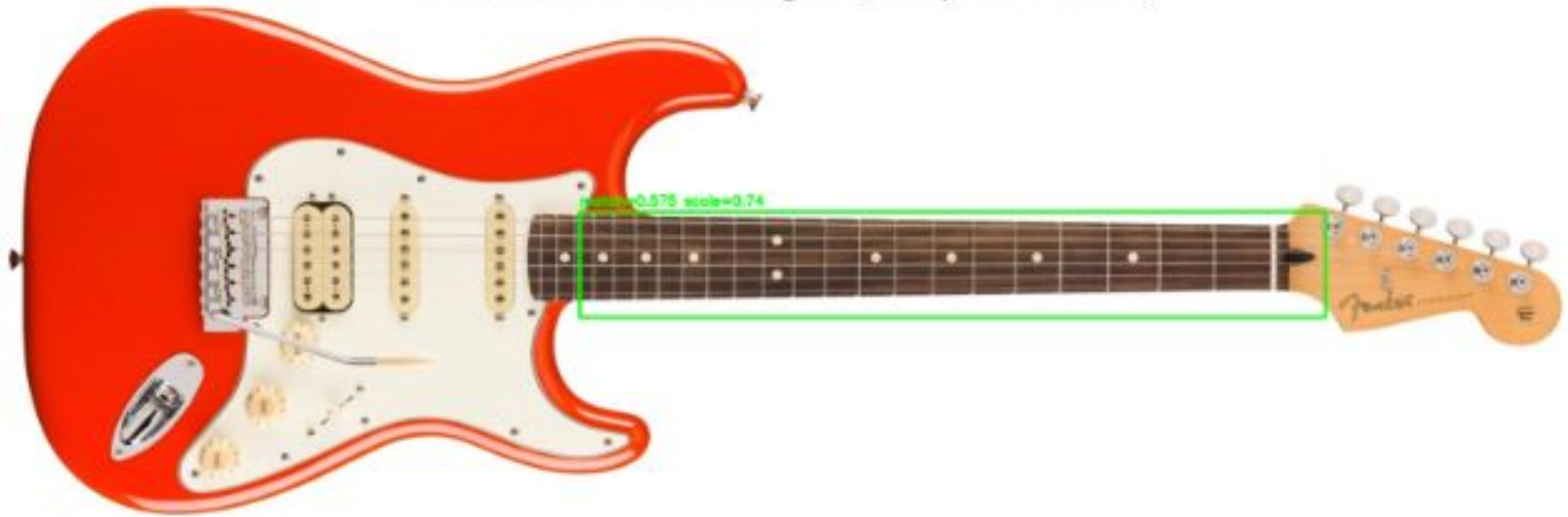


Input edges



Template Matching for Fretboard

Detected fretboard region (template match)



Template Matching for Fretboard

Expanded fretboard ROI



Hough Transforms for fret detection (Related Work)

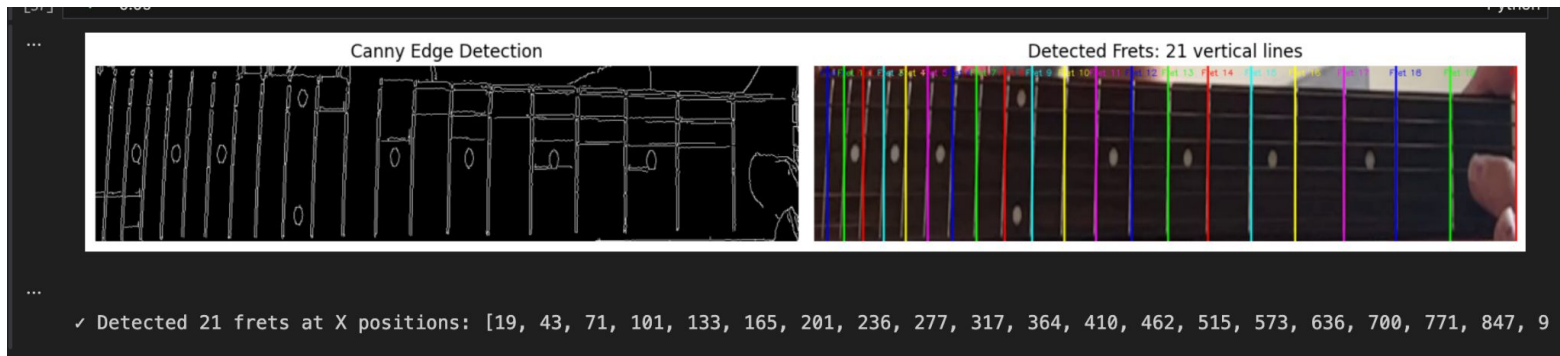
Paleari et al. (2008)

- Canny
- Hough transform
- Orientation alignment
- Projection-based validation
- Audio-visual fusion

Fret-Nav (2025)

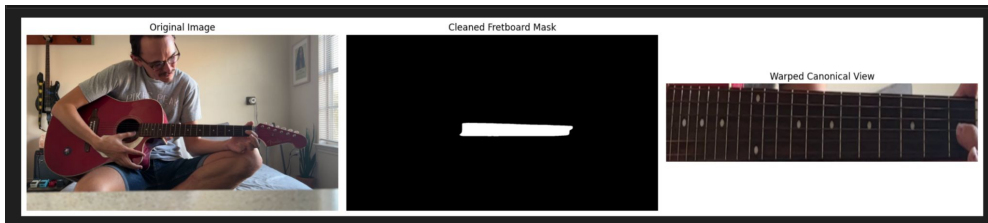
- Hough-based fret detection
 - Fret spacing modelling ($2^{-1/12}$)
 - Real-time feedback system
-

Hough Transforms for fret detection



I used `canonical_fretboard.ipynb` code in the github and tried implement the hough line transforms at the end of this code using the result of this code.

Hough Transforms for fret detection



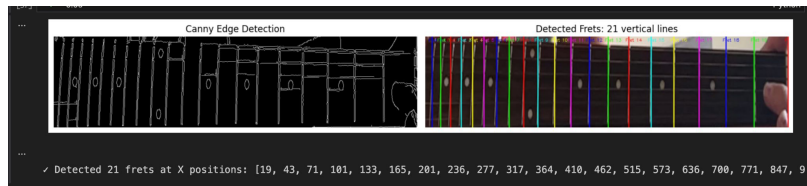
`canonical_fretboard.ipynb` (original code in github)

Mask R-CNN segmentation → `warpPerspective()` → `warped_bgr`



add `detect_frets_in_warped()` using hough transform

Canny → `HoughLinesP` → angle filter → cluster merge



Feb 25 2026 Update

Computer Vision Transcription

Arnav, Vishaal, Om, Sooji

Improved clean_mask() — Fixing Mask Truncation Caused by Hand Occlusion

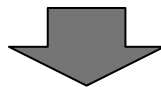
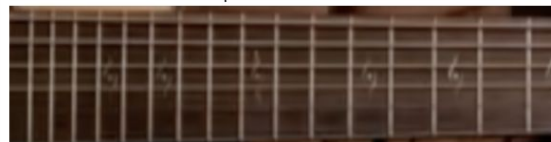
Original Image



Cleaned Fretboard Mask



Warped Canonical View



Original Image



Cleaned Fretboard Mask



Warped Canonical View

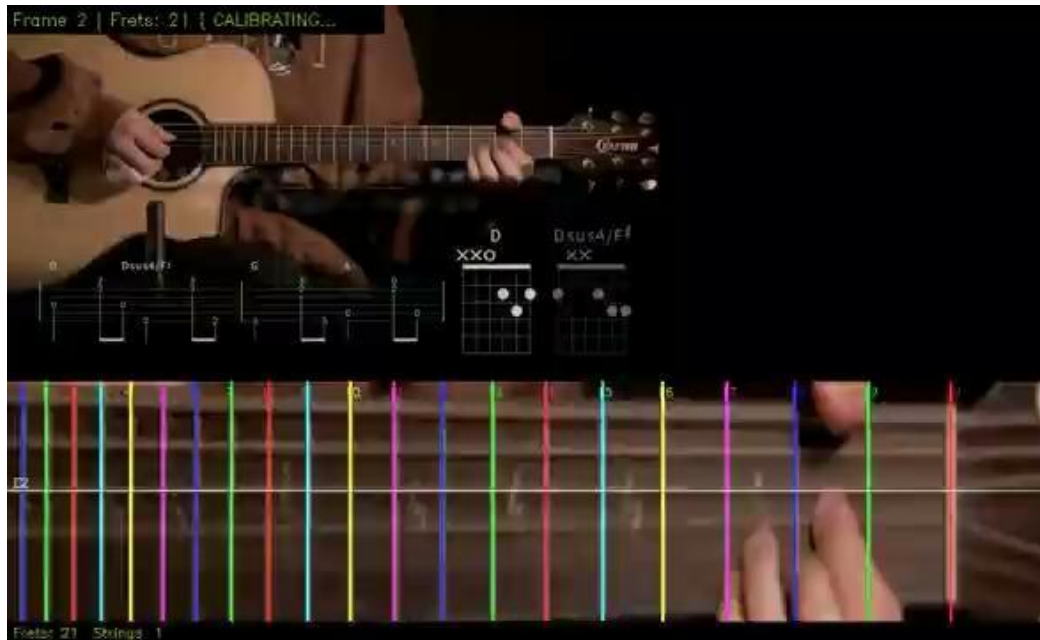
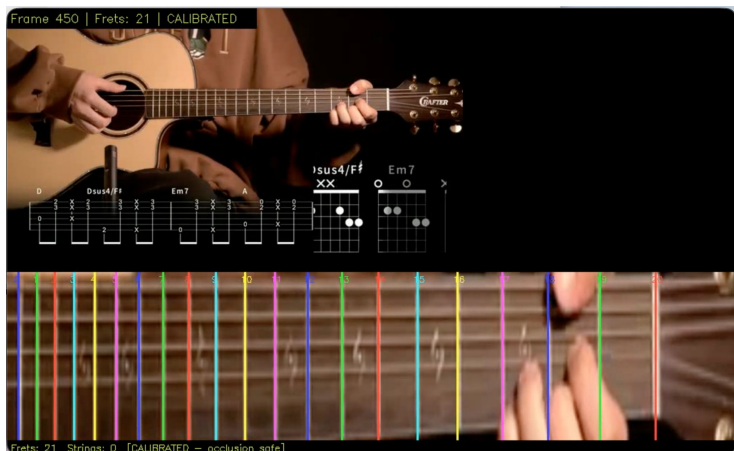


Improved clean_mask() — Fixing Mask Truncation Caused by Hand Occlusion

Mask R-CNN fails to detect the fretboard region covered by the hand, causing the mask to be cut off in the middle.

- Before: only used morphological closing → no effect
 - After: applied horizontal dilation $(150 \times 1) \times 3$ iterations to fill the gap toward the right
 - Left boundary is preserved from the original mask to prevent other objects (e.g. hand) from being included
-

Extended from Static Image → Video Pipeline



2/25 Progress Update

AirStrum: A virtual guitar using real-time hand gesture recognition and strumming technique (Arul, Panda, & Nair 2024):

- Overall Summary:
 - Utilized a CNN model trained on expected chord hand gestures
 - Velocity detection trained on right hand movements to assist in predicting chords
- Potentially valuable:
 - Trained model to recognize hand poses, could assist with the fretboard approach
 - Right hand (strumming hand) detection can also be important in identifying notes

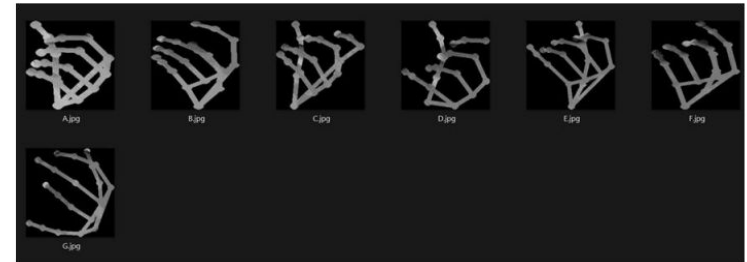
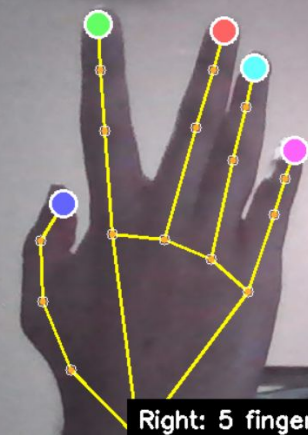


Figure 5. Sample images created for all chord

Finger Tracking Demo for Guitar AMT

Index Depth: 0.019
Total Fingers Extended: 5

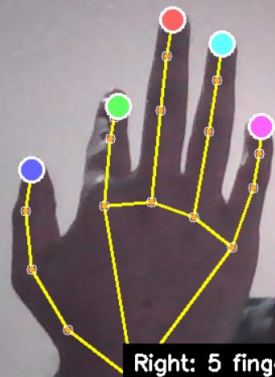


Right: 5 fingers

Show your hands to the camera
Press 'q' to quit

Finger Tracking Demo for Guitar AMT

Index Depth: -0.000
Total Fingers Extended: 5



Right: 5 fingers

Show your hands to the camera
Press 'q' to quit

March 4 2026 Update

Computer Vision Transcription

Arnav, Vishaal, Om, Sooji

Hand/Finger Position Based Chord Detection

Arul et al. (2024)

- CNN
- Analyzed hand skeleton
- Trained on still images
- No actual guitar

Romanian Journal of Information Technology and Automatic Control, Vol. 34, No. 4, 127-139, 2024

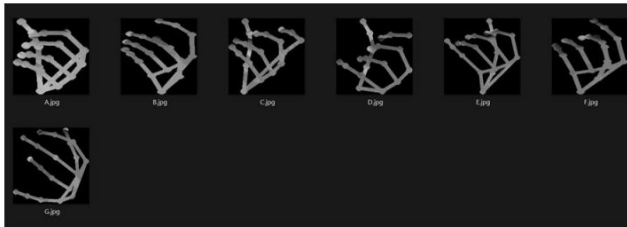


Figure 5. Sample images created for all chord

GuitarPicks (2023)

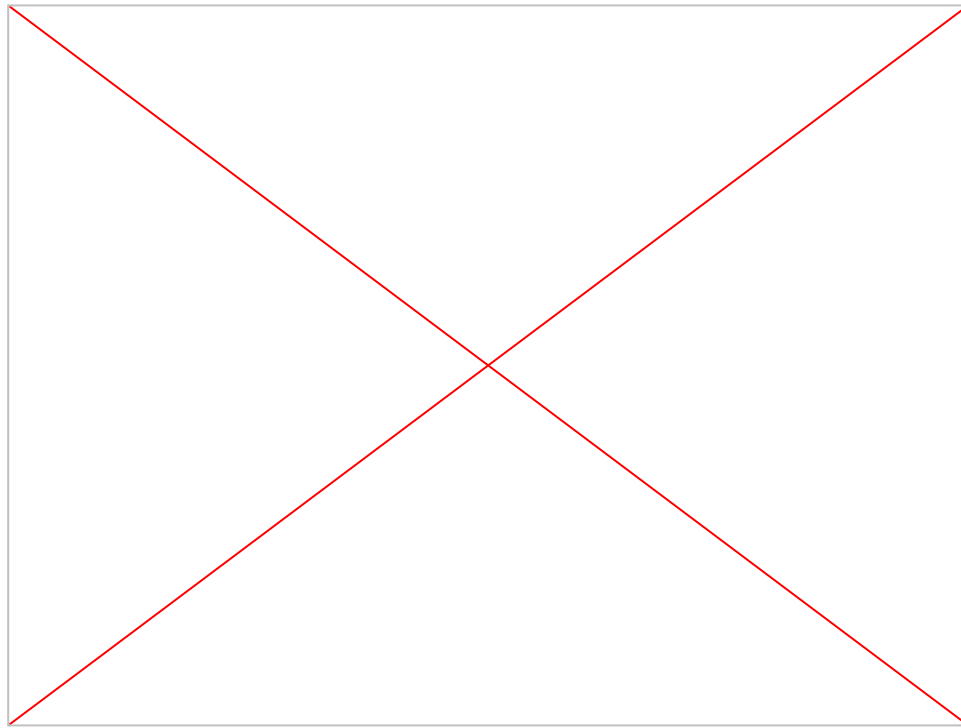
- Customizable gestures
- Landmark based detection
- CV with Mediapipe



Current Progress/Next Steps

- Both code bases depend on MediaPipe Solutions
 - Don't directly address our objective
 - Trained and tested on webcam footage
-

YOLO dim + noisy background demo



Mask

- Retrained Mask-RCNN on Tap2Tab dataset.
 - Achieved Dice score of 0.9071.
-

Fret detection evaluation

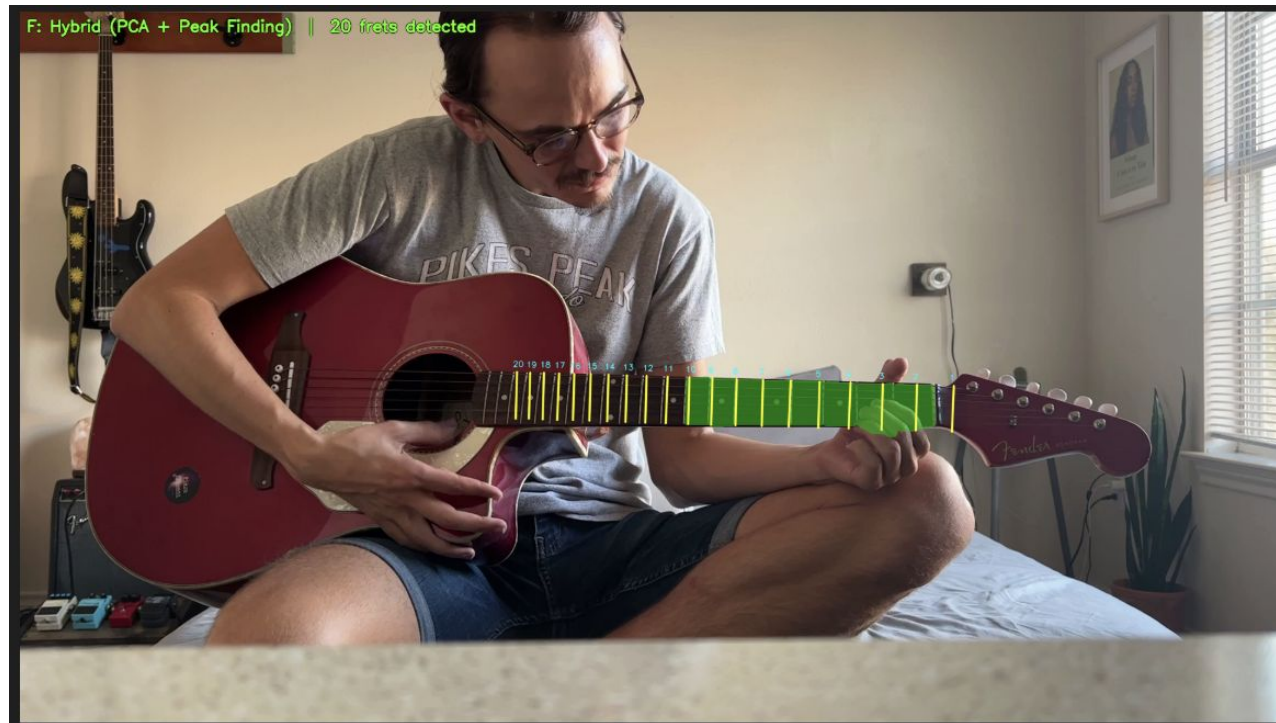
FRET DETECTION EVALUATION RESULTS

	Method	Split	P@0.5%	R@0.5%	F1@0.5%	P@1.0%	R@1.0%	F1@1.0%	P@2.0%	R@2.0%	F1@2.0%	MPE(%w)
A:	Arnav Canon.	test	0.2486	0.2238	0.2355	0.4208	0.3787	0.3987	0.6056	0.5450	0.5737	1.315
A:	Arnav Canon.	valid	0.2152	0.1938	0.2040	0.3899	0.3512	0.3695	0.5949	0.5359	0.5639	1.377
B:	PCA + Warp	test	0.4181	0.3762	0.3961	0.6375	0.5737	0.6039	0.8417	0.7575	0.7974	0.781
B:	PCA + Warp	valid	0.4101	0.3694	0.3887	0.6316	0.5690	0.5987	0.8475	0.7634	0.8032	0.804
C:	PCA (no warp)	test	0.3786	0.3412	0.3590	0.5922	0.5337	0.5615	0.8252	0.7438	0.7824	0.845
C:	PCA (no warp)	valid	0.3693	0.3341	0.3508	0.5854	0.5296	0.5561	0.8173	0.7395	0.7764	0.883
D:	Paleari&Huet	test	0.3233	0.2150	0.2583	0.5658	0.3762	0.4520	0.6823	0.4537	0.5450	0.865
D:	Paleari&Huet	valid	0.3116	0.2155	0.2548	0.5416	0.3746	0.4429	0.6711	0.4641	0.5487	0.949
E:	Paleari Ref.	test	0.3686	0.2437	0.2935	0.5747	0.3800	0.4575	0.6919	0.4575	0.5508	0.903
E:	Paleari Ref.	valid	0.3561	0.2412	0.2876	0.5623	0.3808	0.4541	0.6911	0.4681	0.5581	0.933
F:	Hybrid	test	0.5302	0.2412	0.3316	0.7280	0.3312	0.4553	0.8352	0.3800	0.5223	0.721
F:	Hybrid	valid	0.4773	0.2759	0.3497	0.6568	0.3797	0.4812	0.7742	0.4475	0.5672	0.714

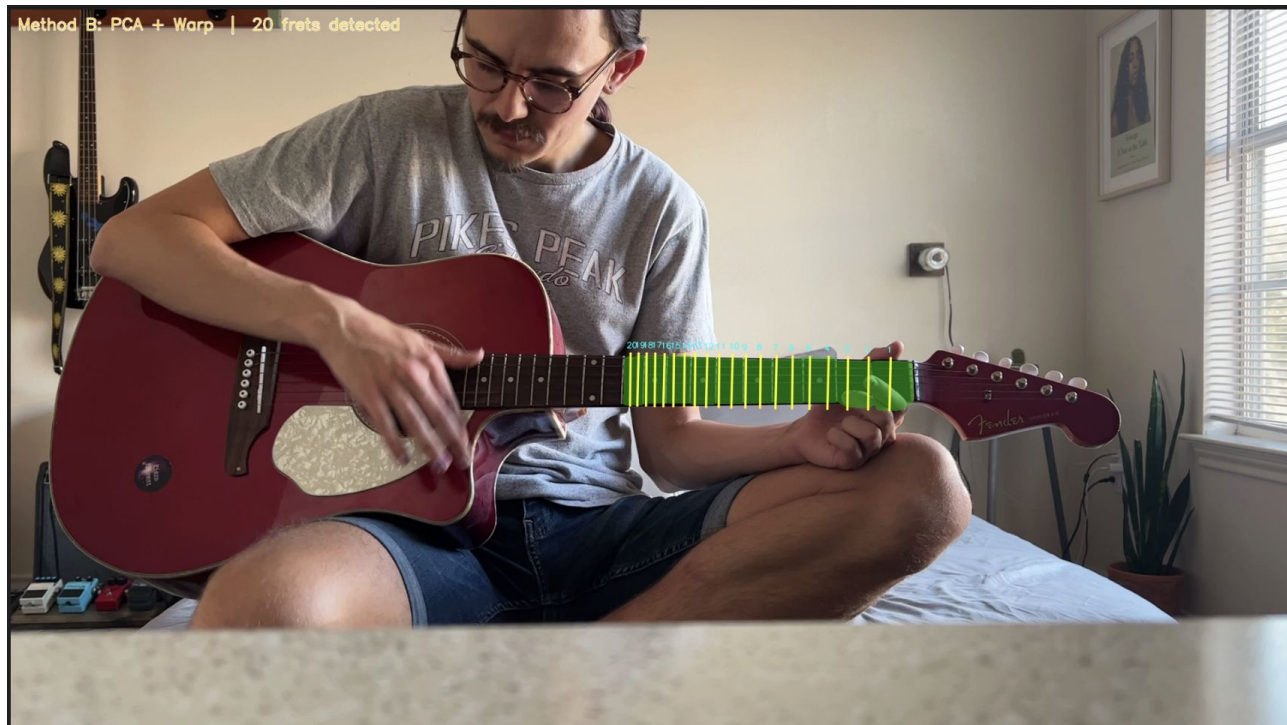
Addressing Mask Limitations

- The evaluation highlighted that when masks miss portions of the guitar, aggressive Rule of 18 extrapolation (predicting all 20 frets unconditionally) causes extreme misalignment.
 - The tuned Method F solves this by treating the geometric warped space solely as a bounding filter for real fret features.
-

Method F



Method B



Fret detection evaluation

- Method B: The PCA + Warp method is the most robust geometric method since fret lines safely narrow towards the bridge and properly fit the bounding geometry of the neck.
 - Method F: achieves the highest strict precision and the lowest Mean Pixel Error. It only predicts frets where actual visual edges are mathematically confirmed by the Sobel filter + mask bounds, preventing "Phantom Frets" in cases where the mask is short.
-

Next Steps

- Formalize pipeline from raw video to canonicalized fretboard
 - Try YOLO and evaluate YOLO.
 - Improve mask so fret detection works as expected.
 - Evaluate Template-based fretboard detection.
 - Evaluate directly training a CNN on Tap2Tab dataset for fret detection.
-

March 11 2026 Update

Computer Vision Transcription

Arnav, Vishaal, Om, Sooji

Occlusion Limitations of MediaPipe and the ViTPose Alternative

Vulnerability to Self-Occlusions

Guitar techniques involve frequent self-occlusions where fingers overlap, causing MediaPipe to produce unstable or inaccurate keypoint predictions

Depth Estimation Ambiguity

MediaPipe provides relative 2.5D coordinates rather than absolute depth, which makes it difficult to distinguish between a finger "hovering" over a string versus "pressing" it

ViTPose: Simple Vision Transformer Baselines for Human Pose Estimation

	AlphaPose	DeepLabCut	Detectron 2	MediaPipe	HRnet BU	HRnet TD	OpenPose	ViTPose
ours-Supine images								
AP	67.7	N/A	52.6	1.8	75.3	59.7	74.4	N/A
AR	74.7	N/A	79.4	6.1	85.1	89.3	79.0	N/A
ours-Supine videos								
AP	67.5	0.0	24.1	1.8	74.6	60.5	55.2	88.5
AR	74.3	0.0	78.7	5.9	84.8	89.2	62.4	90.9
ours-Lap images								
AP	25.0	N/A	11.0	7.8	10.9	20.1	27.0	30.4
AR	57.6	N/A	49.4	20.3	60.1	69.5	67.4	60.7
SyRIP images								
AP	28.0	N/A	25.6	0.0	26.2	30.9	38.4	41.9
AR	50.3	N/A	57.1	0.0	53.8	63.8	54.2	65.4
MINI-RGBD images								
AP	60.5	N/A	48.1	6.1	62.2	59.9	66.5	N/A
AR	69.7	N/A	73.9	17.8	76.7	81.7	72.5	N/A
MINI-RGBD videos								
AP	52.1	3.2	29.5	5.6	53.8	56.8	61.2	73.7
AR	62.7	12.3	68.0	17.4	71.6	77.9	67.5	79.1

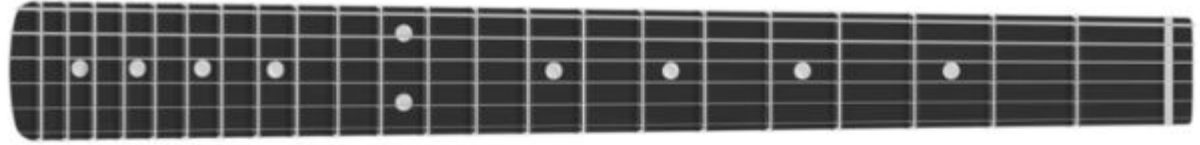
Table 3: Average Precision (AP) and Average Recall (AR) for each method, dataset and input type.

March 25 2026 Update

Computer Vision Transcription

Arnav, Vishaal, Om, Sooji

Template Matching + Fretboard Mask



Input



Template Matching + Fretboard Mask

Expanded fretboard ROI

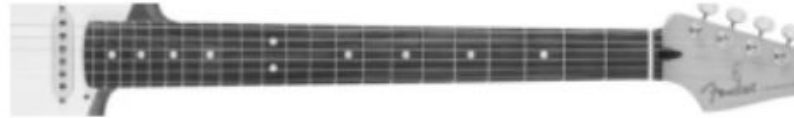


Template Matching + Fretboard Mask

ROI



ROI gray



ROI mask



Template Matching + Fretboard Mask

ROI edge fit



Template Matching + Fretboard Mask

ROI edge fit



ROI fret detection



YOLO vs Mask R-CNN

In terms of Segmentation quality:

- Dice Score: YOLOv8n-seg (0.9009) vs. Mask R-CNN (0.9071) – Effectively identical accuracy
 - Variance: YOLOv8n-seg (± 0.1396) vs. Mask R-CNN (± 0.2167) – YOLO boundary masks are more stable
-

Fret Detection Improvements

Best Overall Method: PCA + Warp paired with YOLO masks outperformed the Mask R-CNN baseline.

F1 @ 2% Threshold: YOLO (0.8067) vs. Mask R-CNN (0.7974)

Mean Pixel Error: 0.788% of image width

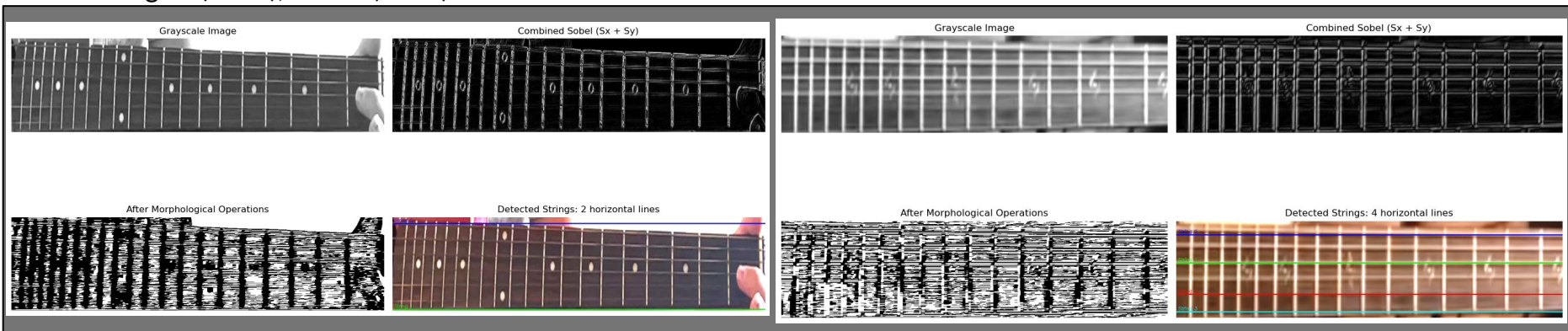
At 1% Threshold:

- Method C (PCA no warp): YOLO (0.6365) vs. Mask R-CNN (0.5615) -> +7.5% jump in F1 score
 - Method B (PCA + warp): YOLO (0.6333) vs. Mask R-CNN (0.6039) -> +2.9% jump in F1 score
-

String detection

Baseline: Sobel H+V + Hough

Duke & Salgian (2019), Asmar (2022)



Grayscale conversion -> **Sobel filter** ($H + V$, kernel=3, weight 0.5 each) -> **Adaptive threshold** (mean of 5px neighborhood - 2)
-> **Morphological opening** (1×5 kernel) -> **Morphological closing** (1×7 kernel) -> **Hough Line Transform**

remove colors -> find edges -> remain only black or white

-> remove noise -> connect the lines into one line -> find the lines

Method	image1	image2	Better under
Baseline (Sobel H+V)	4 / 6	2 / 6	Good light
Method A (Projection)	1 / 6	4 / 6	Poor light
Method B (Hough + K-Means)	6 / 6	6 / 6	Both
Method C (H Sobel only)	6 / 6	3 / 6	Good light

Method A (Projection)

Grayscale conversion -> Horizontal projection -> Peak detection (`scipy.signal.find_peaks`) -> Draw horizontal lines at each peak Y position

Method B (Hough + K-Means)

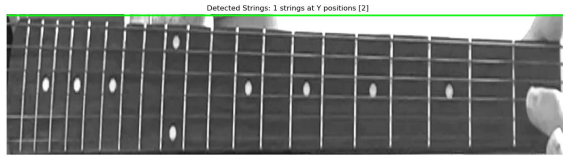
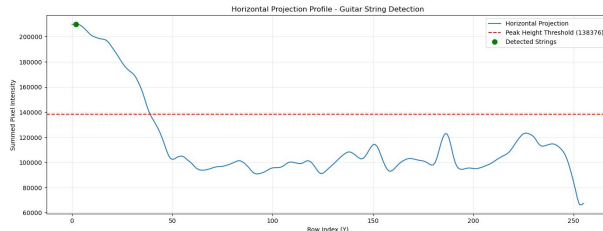
baseline (with wide kernel) -> Filter near-horizontal lines only ($\pm 10^\circ$) -> Compute Y value of each line at image center X -> K-Means clustering with $n=6$

Method C (H Sobel only)

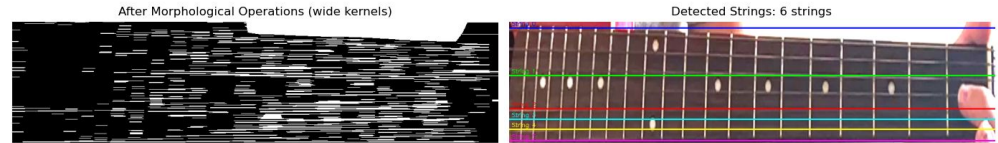
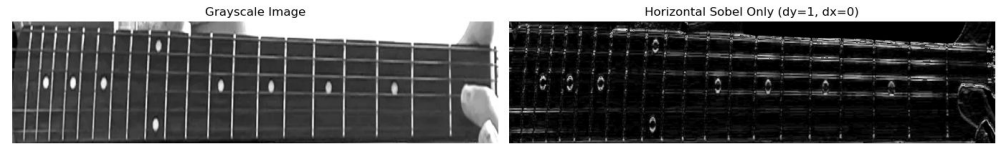
Same with baseline with Horizontal Sobel ONLY

String detection (image1)

Method A: horizontal projection profile



Method C: horizontal Sobel only



Method B: Hough + K-Means (6 clusters)

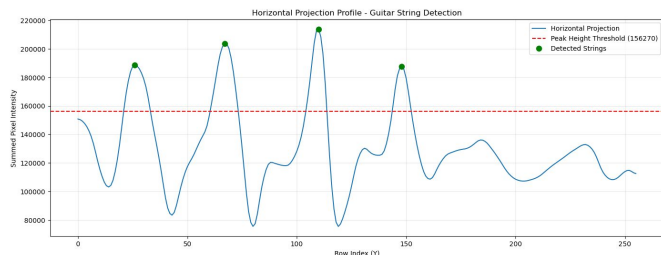


K-Means String Detection: 6 Strings Detected

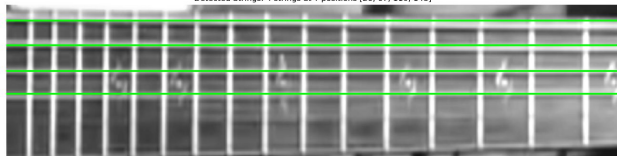


String detection (image2)

Method A: horizontal projection profile

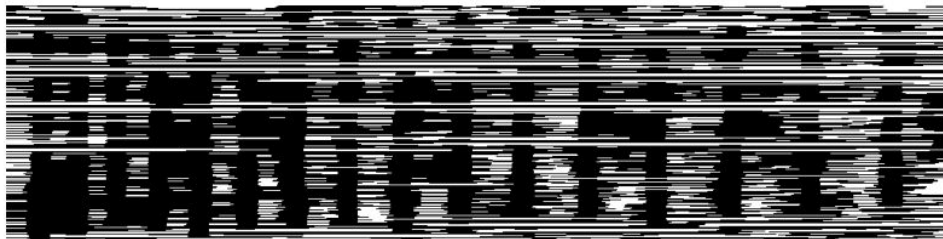


Detected Strings: 4 strings at Y positions [26, 67, 110, 148]



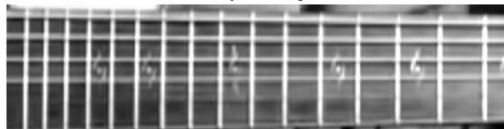
Method B: Hough + K-Means (6 clusters)

After Morphological Operations

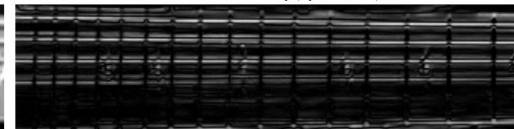


Method C: horizontal Sobel only

Grayscale Image



Horizontal Sobel Only (dy=1, dx=0)



After Morphological Operations (wide kernels)



Detected Strings: 3 strings



K-Means String Detection: 6 Strings Detected



Recap:

Arul et al. (2024)

- CNN
- Analyzed hand skeleton
- Trained on still images
- No actual guitar

Romanian Journal of Information Technology and Automatic Control, Vol. 34, No. 4, 127-139, 2024

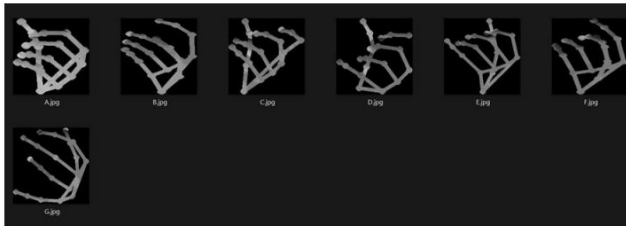


Figure 5. Sample images created for all chord

GuitarPicks (2023)

- Customizable gestures
- Landmark based detection
- CV with Mediapipe



Progress Update:

- Working version of GuitarPicks approach
- Training on A, C, and G chords
- Utilizing CV and landmark based detection to train the cnn model



Current Work / Next Steps:

- ViTPOSE paper could complement this approach
 - Train and test on other chords
 - Test on video/better data
 - Evaluate Template Matching + Fretboard Mask
 - Train mask and fret detection on more data
-

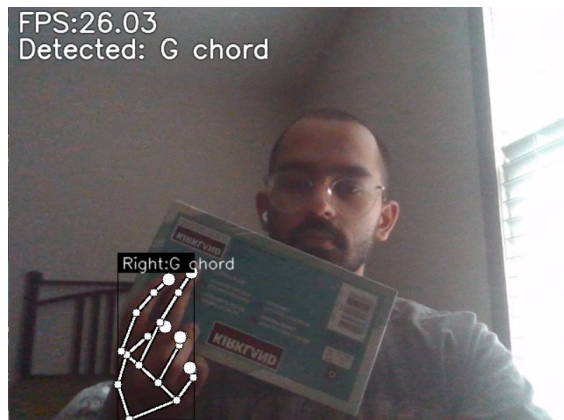
April 1st 2026 Update

Computer Vision Transcription

Arnav, Vishaal, Om, Sooji

MediaPipe Hand Detection

- Working version of GuitarPicks approach
- Utilizing CV and landmark based detection to train the cnn model
- Added image and video processing options to the pipeline



Evaluating Template Matching + fretboard mask for fret detection

- Template Matching achieves the best positional accuracy of any method with Mean Pixel Error (MPE) of 0.716%.
 - However, it has lower recall than base PCA + warp which remains the best overall method with $F1@2\% = 0.8067$.
 - Template matching is great at placing frets accurately but misses frets that other methods find.
-

Updated Ranking

Overall Method Ranking (by F1@2.0%, Test Split)

Rank	Method	Mask R-CNN	YOLO
1	B: PCA + Warp	0.7974	0.8067
2	C: PCA (no warp)	0.7824	0.7963
3	A: Arnav Canon.	0.5737	0.5907
4	E: Paleari Ref.	0.5508	0.5589
5	D: Paleari & Huet	0.5431	0.5597
6	F: Hybrid	0.5092	0.4720
7	G: Templ.Match	0.4255	0.4055

By Best MPE (Positional Accuracy)

Rank	Method	Mask R-CNN	YOLO
1	G: Templ.Match	0.785	0.716
2	F: Hybrid	0.738	0.735
3	B: PCA + Warp	0.774	0.788
4	C: PCA (no warp)	0.845	0.762

OLD method for fret detection

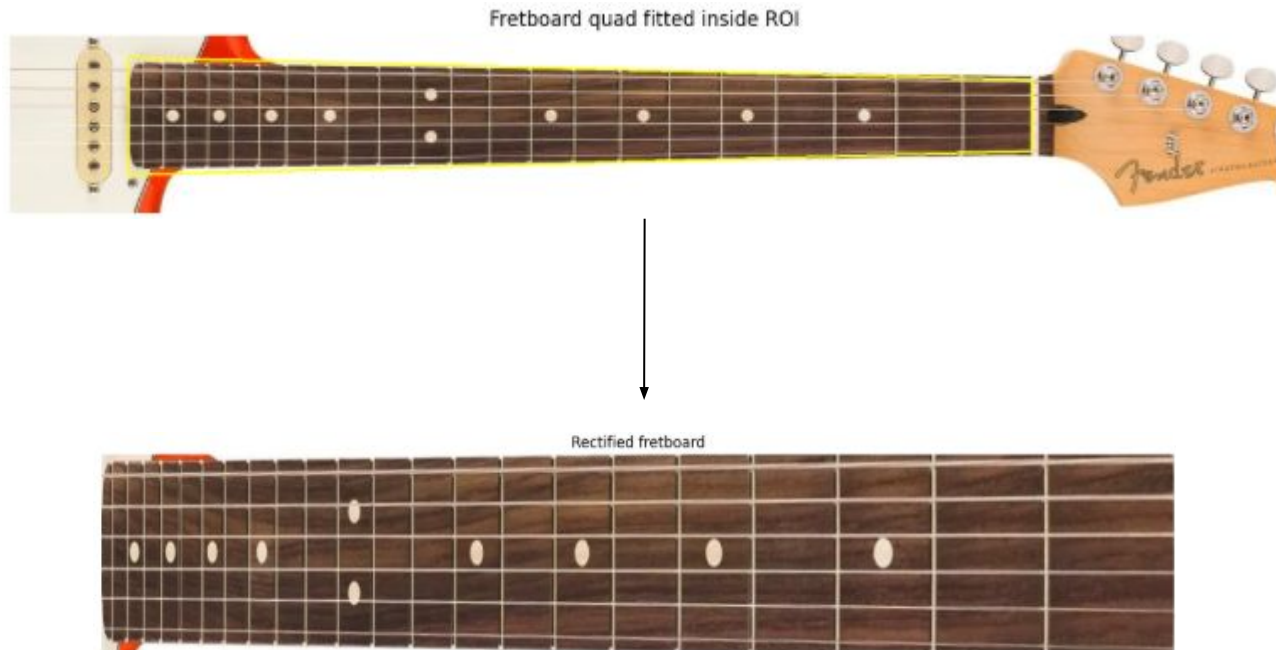
ROI edge fit



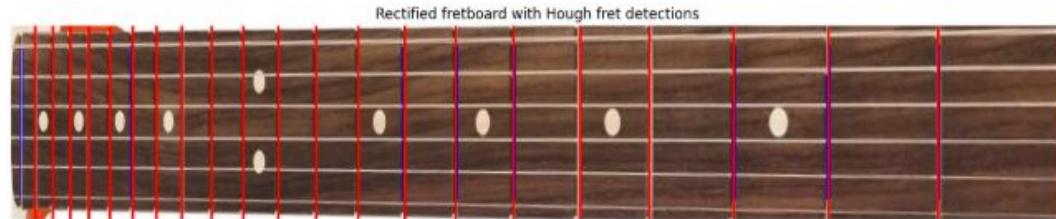
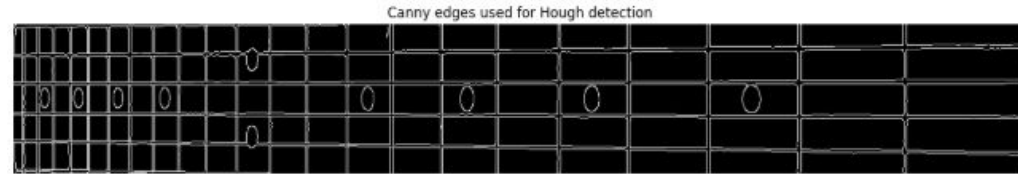
ROI fret detection



Hough Transforms for Fret Detection



Hough Transforms for Fret Detection



Hough Transforms for Fret Detection

Expanded fretboard ROI



Cropped fretboard ROI



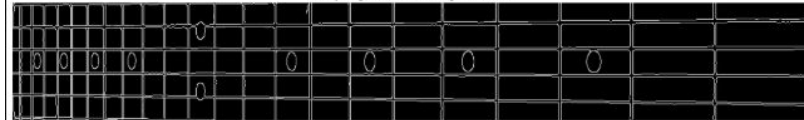
ROI fretboard mask



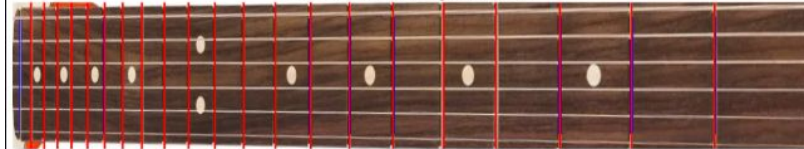
Fretboard quad fitted inside ROI



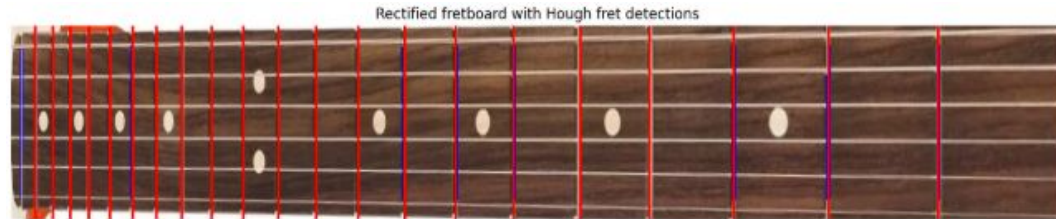
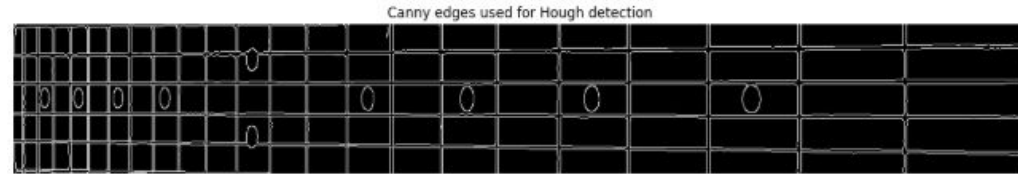
Canny edges used for Hough detection



Rectified fretboard with Hough fret detections



Hough Transforms for Fret Detection



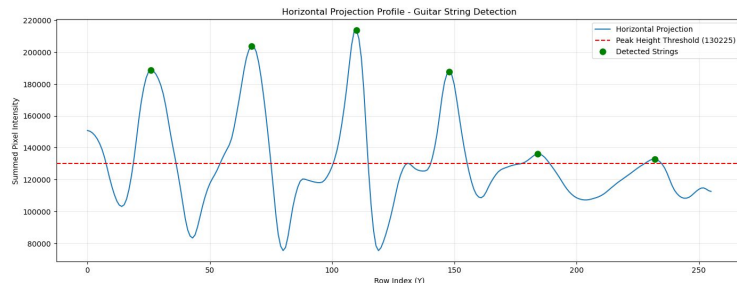
Hough Transforms for Fret Detection



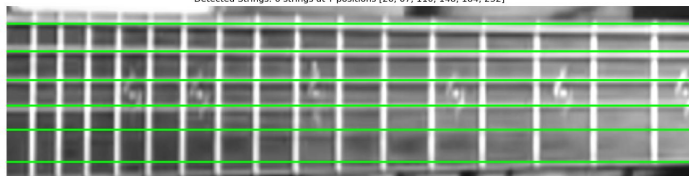
Roadblocks / Next Steps:

- Data format is a bottleneck
 - Incorporate string detection methods into the overall pipeline
-

String detection



Detected Strings: 6 strings at Y positions [26, 67, 110, 148, 184, 232]



After Morphological Operations



Problem with k-mean method:

Hand on fretboard → creates extra Hough line segments along hand boundaries

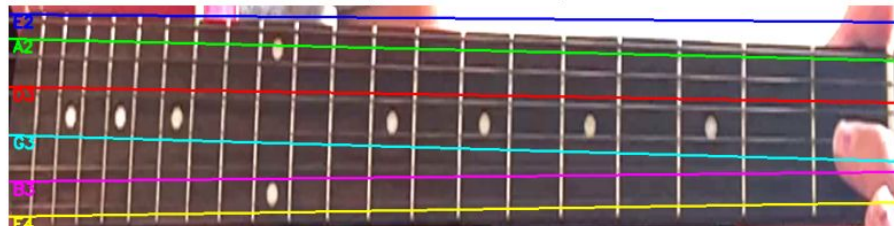
K-Means clusters ALL detected lines including hand edges

Result: string positions get pulled and tilted toward the hand

K-Means is line-based → vulnerable to hand edges.

Projection is row-based → hand position does not affect string Y coordinates.

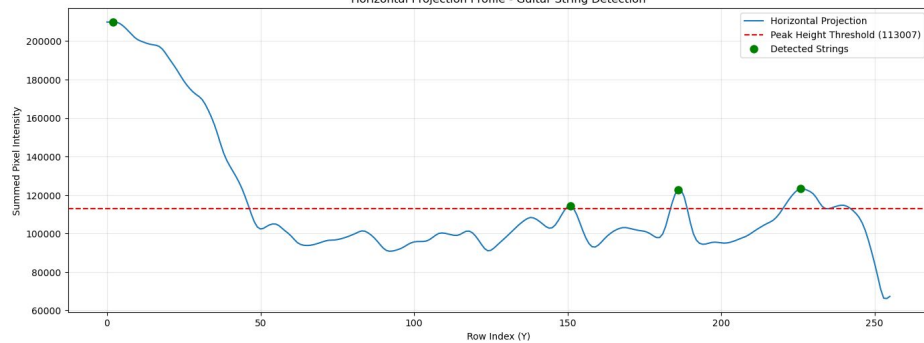
K-Means String Detection: 6 Strings Detected



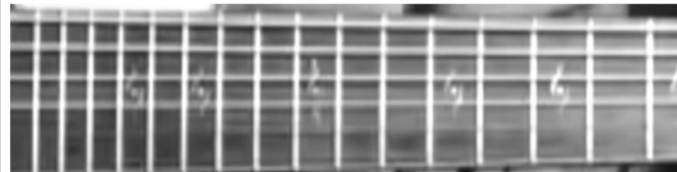
Grayscale Image



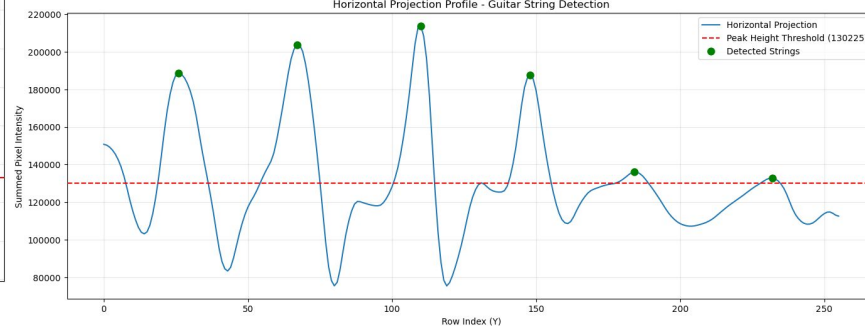
Horizontal Projection Profile - Guitar String Detection



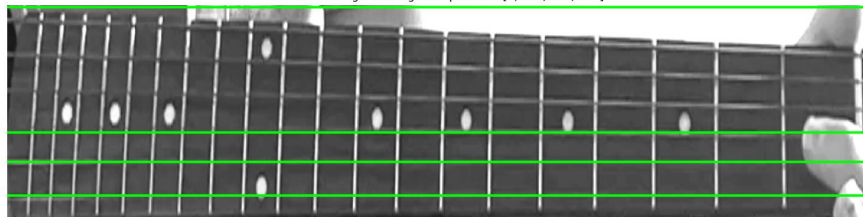
Grayscale Image



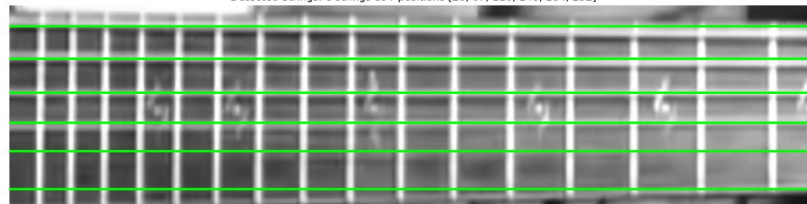
Horizontal Projection Profile - Guitar String Detection



Detected Strings: 4 strings at Y positions [2, 151, 186, 226]



Detected Strings: 6 strings at Y positions [26, 67, 110, 148, 184, 232]



Method	image1	image2	Better under
Baseline (Sobel H+V)	4 / 6	2 / 6	Good light
Method A (Projection)	1 / 6	4 / 6	Poor light
Method B (Hough + K-Means)	6 / 6	6 / 6	Both
Method C (H Sobel only)	6 / 6	3 / 6	Good light

Method A (Projection)

Grayscale conversion -> Horizontal projection -> Peak detection (`scipy.signal.find_peaks`) -> Draw horizontal lines at each peak Y position

Method B (Hough + K-Means)

baseline (with wide kernel) -> Filter near-horizontal lines only ($\pm 10^\circ$) -> Compute Y value of each line at image center X -> K-Means clustering with $n=6$

Method C (H Sobel only)

Same with baseline with Horizontal Sobel ONLY

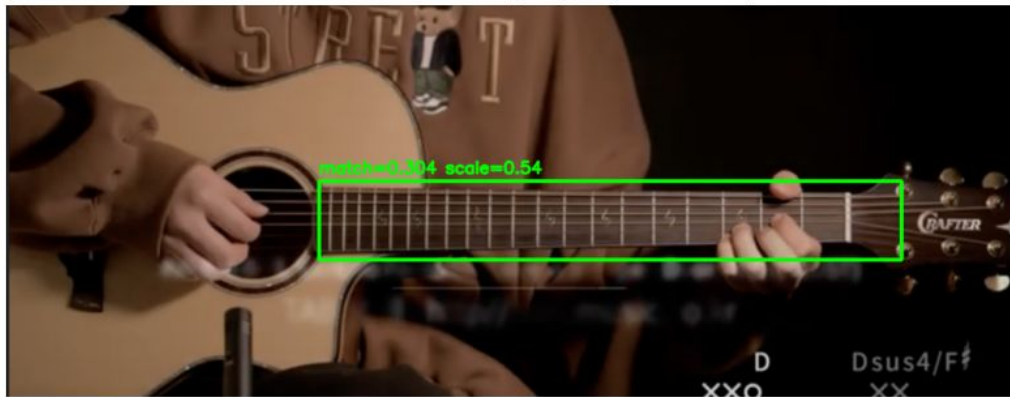
April 8 2026 Update

Computer Vision Transcription

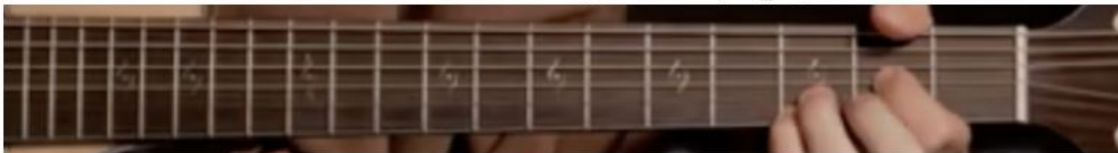
Arnav, Vishaal, Om, Sooji

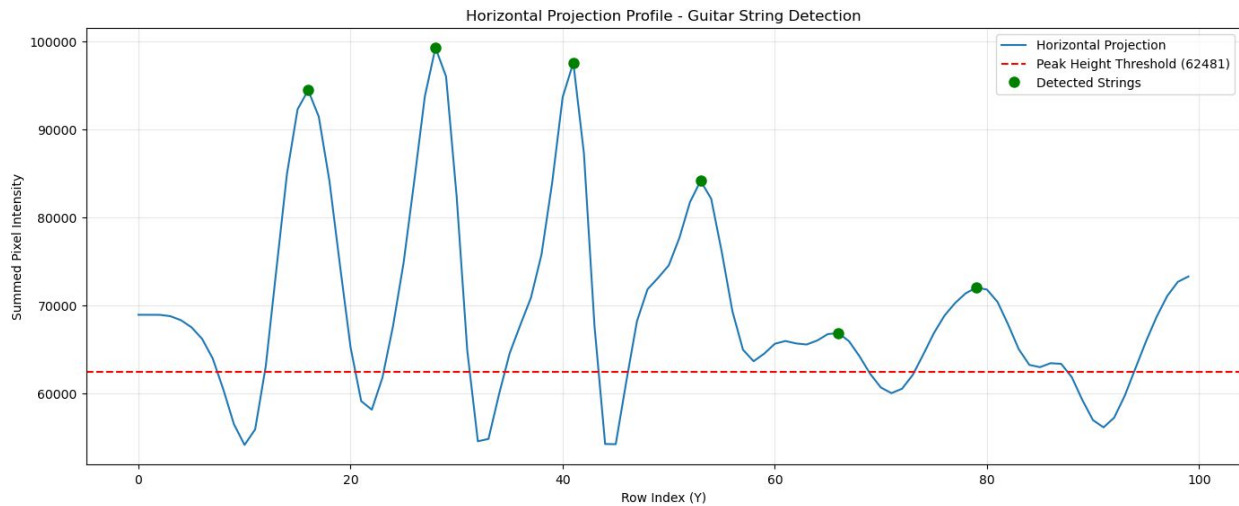
integrated with template matching and string detection

Detected fretboard region (template match)



Detected Fretboard Region → warped_bgr





Detected Strings: 6 strings at Y positions [16, 28, 41, 53, 66, 79]



After Morphological Operations



K-Means String Detection: 6 Strings Detected



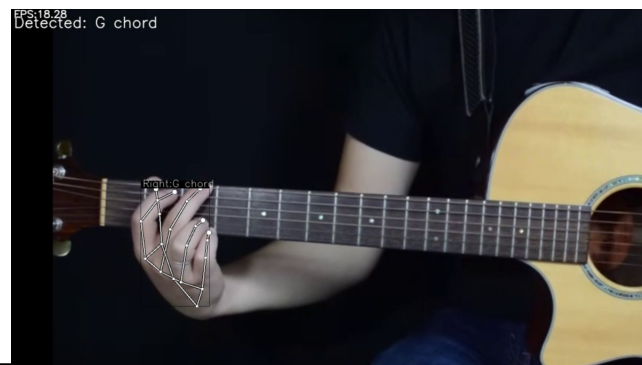
April 15 2026 Update

Computer Vision Transcription

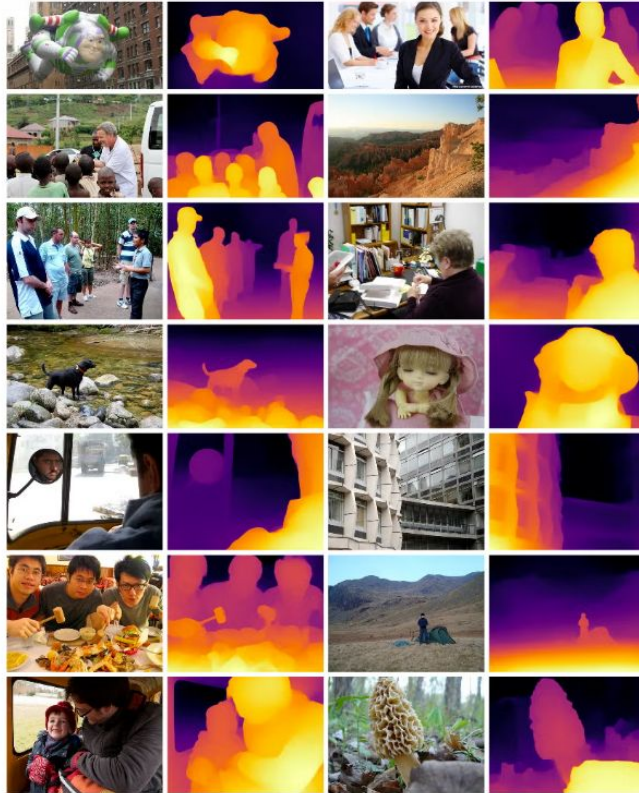
Arnav, Vishaal, Om, Sooji

Hand Tracking / Chord Classification

- Needed video data
- AirStrum and GuitarPicks frameworks had pipelines
- Modified data collection pipelines for video

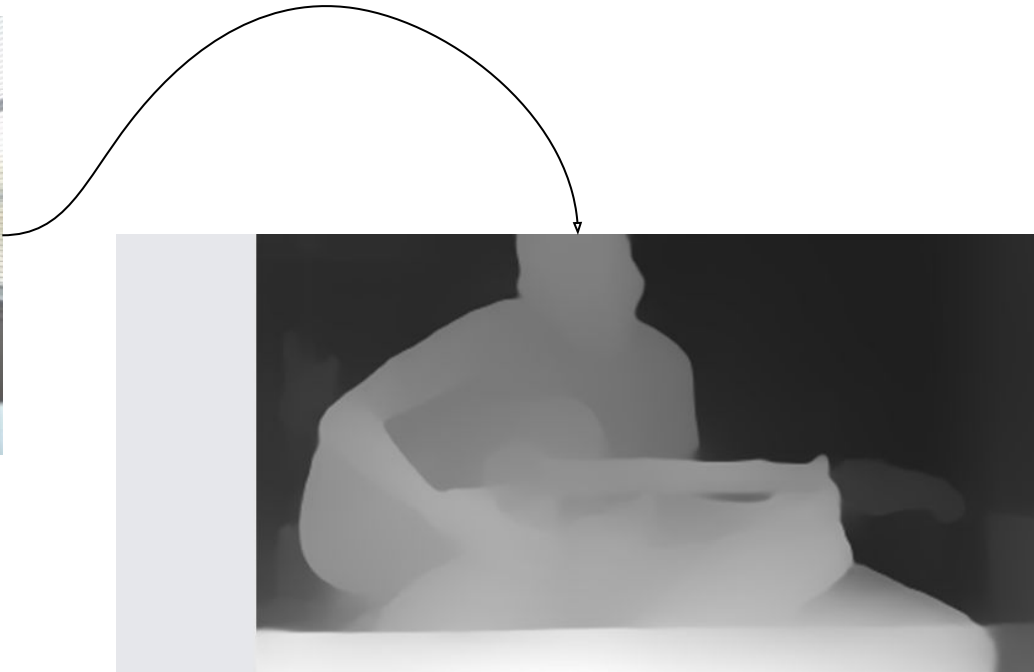


Ranftl et al. (2019) - MiDaS Depth Estimation

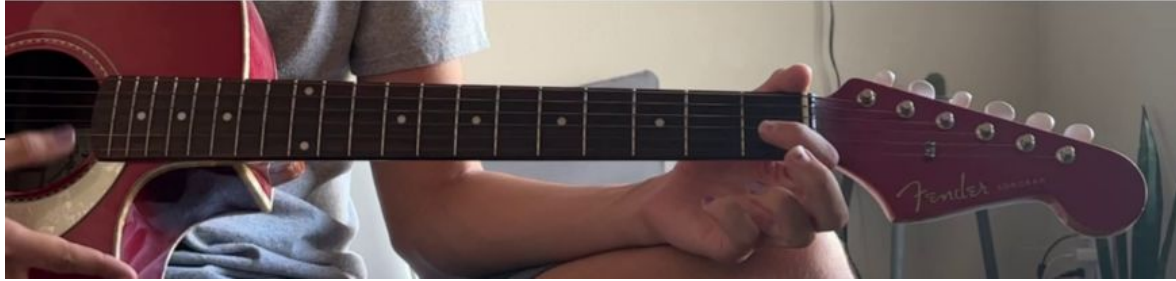


- Paper from 2019 regarding **monocular** depth estimation
 - Most guitar tabs papers used **stereo** depth estimation
-

MiDaS Demo - Full Frame



MiDaS Demo - Fretboard ROI



Arnab et al. (2021)

- Video processed by extracting non-overlapping, 3D spatio-temporal "tubelets" directly from the input volume.
 - Instead of tracking a fingertip coordinate on a 2D plane, a tubelet embedding fuses spatial and temporal information simultaneously. By bypassing explicit pose estimation or bounding boxes, the transformer can learn the direct visual relationship between complex chord fingerings and strumming hand motions directly from the raw pixels.
-

Transcription in Arnab et al.

- The Spatial Encoder first models interactions within the same temporal index (a single frame). For transcription, this extracts the static hand pose and fretboard geometry.
 - The Temporal Encoder then models interactions between those time steps.
 - The paper demonstrates that pure-transformer models can be trained effectively on smaller datasets by stacking specific regularization methods originally developed for CNNs.
-

Next Steps

- Look into more advanced monocular depth estimation methods
 - Evaluate the string detection method on additional datasets with varying lighting conditions and hand occlusions on the fretboard.
-

April 22 2026 Update

Computer Vision Transcription

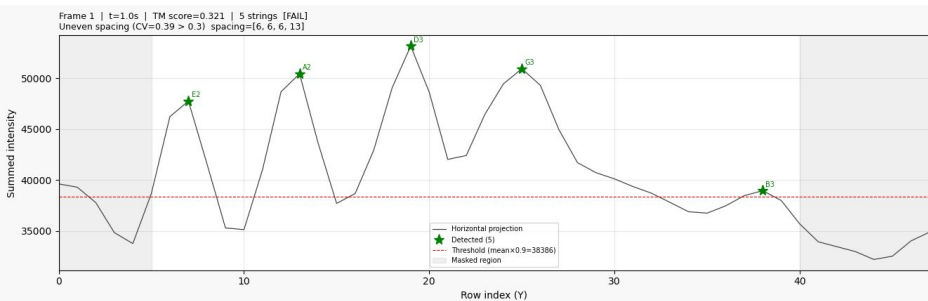
Arnav, Vishaal, Om, Sooji

String detection original (projection)

Frame	Time(s)	TM Score	#Strings	Spacing OK	PASS	TM ms	SD ms	Total ms
0	0	0.000000	5	False	False	179	1	180
1	1	1.000000	5	False	False	164	0	165
2	2	2.000000	5	False	False	165	0	166
3	3	3.000000	6	True	True	197	0	197
4	4	4.000000	5	False	False	164	0	164
5	5	5.000000	4	True	False	165	0	166
6	6	6.000000	5	False	False	165	0	165
7	7	7.000000	4	True	False	192	0	193
8	8	8.000000	5	False	False	217	0	218
9	9	9.000000	4	True	False	172	0	173
10	10	10.000000	4	True	False	174	0	174
11	11	11.000000	6	True	True	166	0	166
12	12	12.000000	4	True	False	180	0	181
13	13	13.000000	4	True	False	182	0	183
14	14	14.000000	5	True	False	180	0	181
15	15	15.000000	4	True	False	204	0	204
16	16	16.000000	5	True	False	213	0	213
17	17	17.000000	4	True	False	260	0	260
18	18	18.000000	4	True	False	253	0	253
19	19	19.000000	4	True	False	191	0	191
20	20	20.000000	4	True	False	214	0	215
21	21	21.000000	5	True	False	181	0	182
22	22	22.000000	5	False	False	178	0	178
23	23	23.000000	4	True	False	176	0	176
24	24	24.000000	4	True	False	177	0	178
25	25	25.000000	6	True	True	199	0	199
26	26	26.000000	6	True	True	184	0	185

```
Method      : Original Projection (threshold = mean × 0.9)
Y-range mask : rows ≤ 5 and rows ≥ H - 8
Total frames : 50
6-string rate : 14.0% (7/50)
Spacing OK rate: 84.0% (42/50)
PASS rate    : 14.0% (7/50)
Avg TM time  : 180.1 ms
Avg SD time  : 0.3 ms
Avg total time : 180.4 ms
```

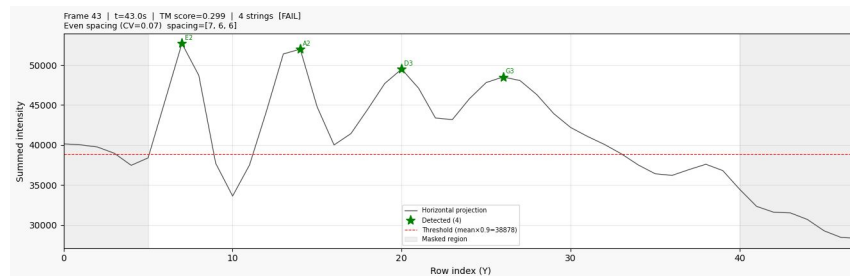
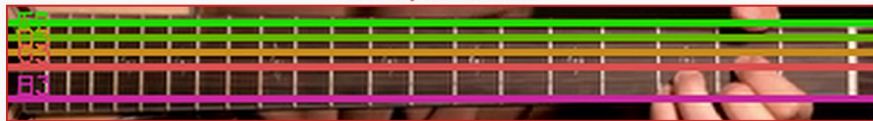
String detection original (projection)



Original ROI (no annotations)



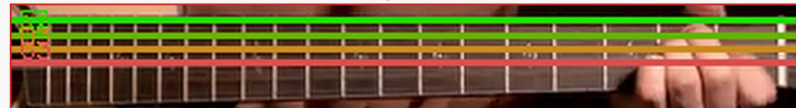
Detected strings [FAIL] [7, 13, 19, 25, 38]



Original ROI (no annotations)



Detected strings [FAIL] [7, 14, 20, 26]



What changed

Auto-Tune Peak Detection

- Before: Fixed threshold ($\text{mean} \times 0.9$) - requires manual tuning per frame
- After: Prominence-based top-6 peak selection (no parameter tuning needed)

Missing String Interpolation

- Before: Fewer than 6 strings detected -> FAIL
 - After: Infers missing string positions from the spacing pattern of detected strings -> recovers 4-string frames to 6
-

String detection updates (compare with k-means)

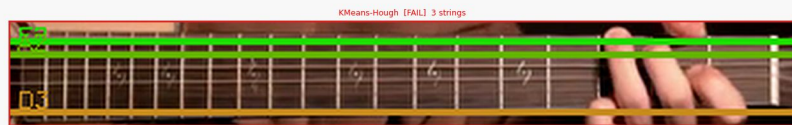
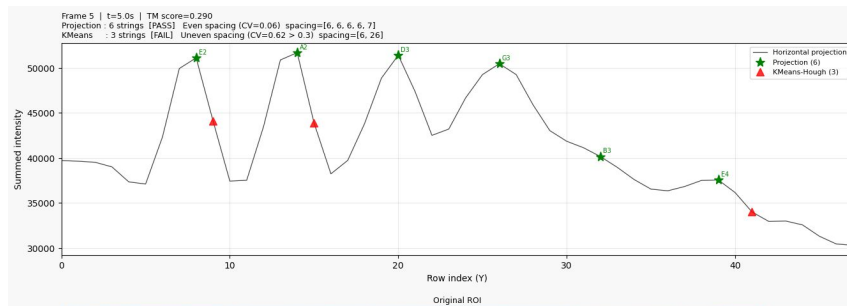
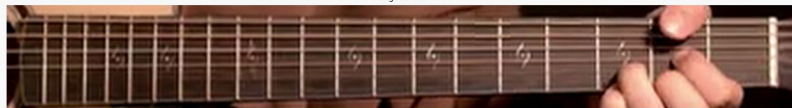
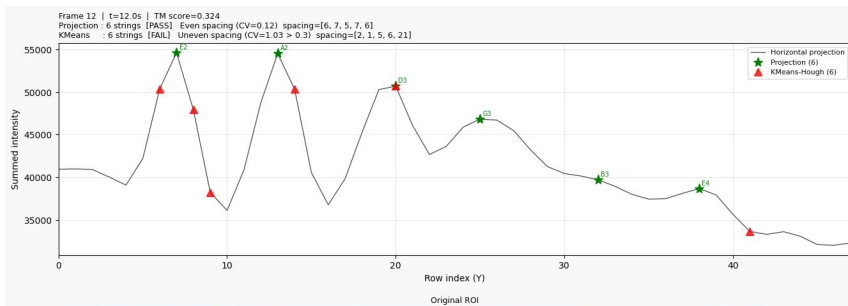
vid_test — Per-Frame Method Comparison (PASS = 6 strings AND even spacing)											
Frame	Time(s)	TM Score	Proj #str	Proj Spacing	Proj PASS	Proj ms	KM #str	KM Spacing	KM PASS	KM ms	
0	0	0.000000	0.319	6	True	True	0	6	False	False	8
1	1	1.000000	0.321	6	True	True	0	6	False	False	5
2	2	2.000000	0.318	6	True	True	0	6	False	False	4
3	3	3.000000	0.324	6	True	True	0	6	False	False	5
4	4	4.000000	0.312	6	True	True	0	5	False	False	5
5	5	5.000000	0.290	6	True	True	0	3	False	False	5
6	6	6.000000	0.304	6	True	True	0	6	False	False	8
7	7	7.000000	0.308	6	True	True	0	5	False	False	5
8	8	8.000000	0.301	6	True	True	0	5	False	False	6
9	9	9.000000	0.313	6	True	True	0	6	False	False	7
10	10	10.000000	0.319	6	True	True	0	3	False	False	4
11	11	11.000000	0.286	6	True	True	0	6	False	False	5
12	12	12.000000	0.324	6	True	True	0	6	False	False	4
13	13	13.000000	0.301	6	True	True	0	6	False	False	8
14	14	14.000000	0.282	6	True	True	0	6	False	False	6
15	15	15.000000	0.289	6	True	True	0	5	False	False	5
16	16	16.000000	0.274	5	True	False	0	6	False	False	6
17	17	17.000000	0.291	6	True	True	0	6	False	False	6
18	18	18.000000	0.290	6	True	True	0	4	False	False	4
19	19	19.000000	0.297	6	True	True	0	6	False	False	4
20	20	20.000000	0.296	6	True	True	0	6	False	False	6
21	21	21.000000	0.280	5	True	False	0	5	False	False	7
22	22	22.000000	0.301	6	True	True	0	6	False	False	5
23	23	23.000000	0.275	6	True	True	0	3	False	False	5
24	24	24.000000	0.283	6	True	True	0	5	False	False	6
25	25	25.000000	0.283	6	True	True	0	6	False	False	6
26	26	26.000000	0.276	6	True	True	0	6	False	False	5
27	27	27.000000	0.312	6	True	True	0	6	False	False	8
28	28	28.000000	0.303	6	True	True	0	6	False	False	7

	Total Frames	6-string rate	Spacing OK rate	PASS rate	Avg time (ms)
Method					
Projection (Auto-Tune + Interpolation)	50	96.0%	100.0%	96.0%	0.2
KMeans-Hough (Asmar 2022)	50	74.0%	0.0%	0.0%	5.8

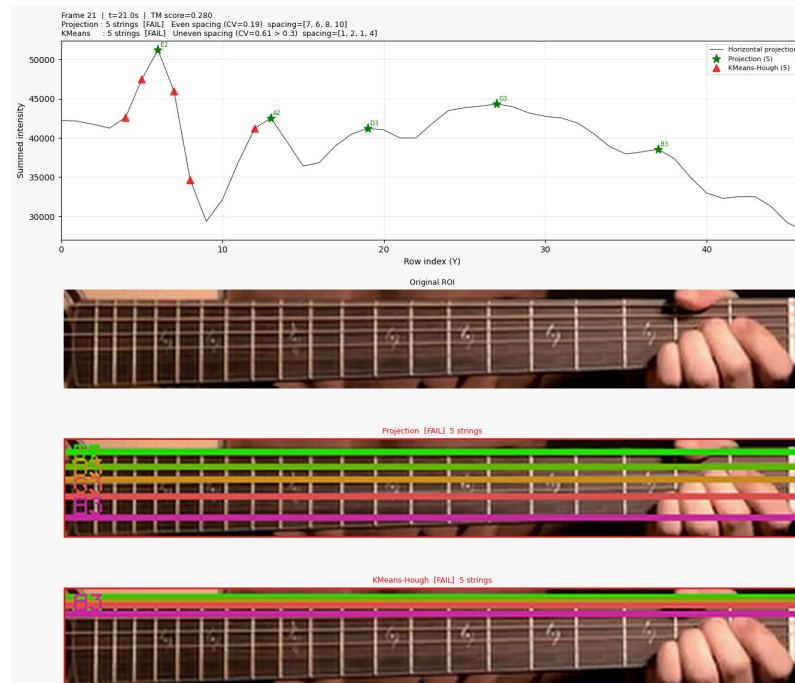
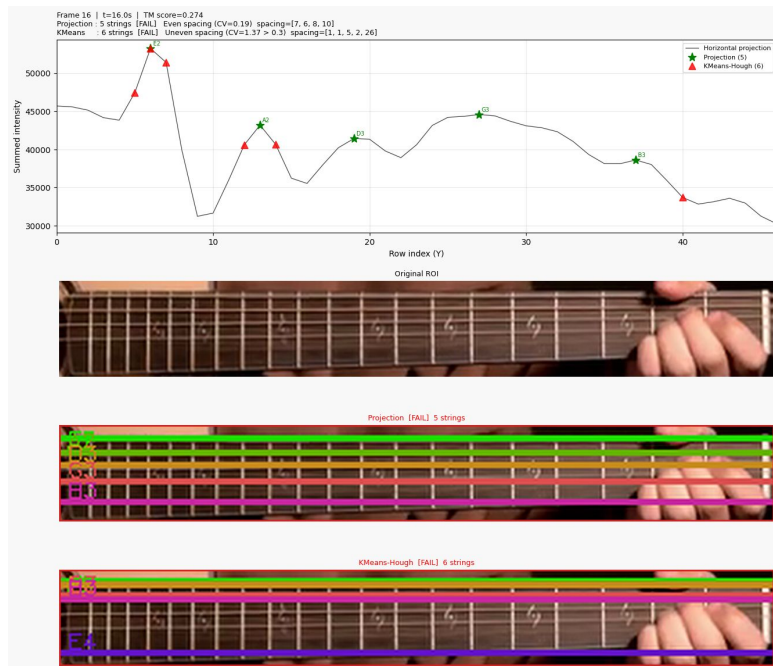
Method agreement analysis (total 50 frames):

Both PASS	:	0 (0.0%)
Projection only	:	48 (96.0%)
KMeans only	:	0 (0.0%)
Both FAIL	:	2 (4.0%)

14% -> 96%



Fail case



ViViT-based approach

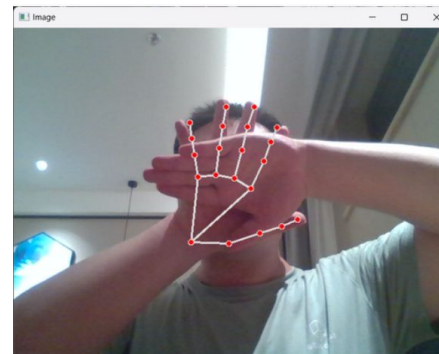
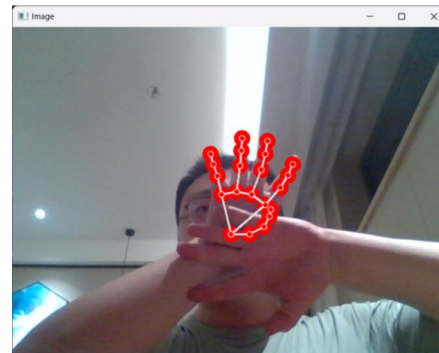
- Canonical warping -> 3D Tubelet Embedding -> Positional encoding including frets -> Factorized Spatial/Temporal attention -> CNN
 - Canonical warping: Standardizes orientation to remove perspective and rotational distortion.
 - 3D Tubelet embedding: Resolves motion blur
-



The Gesture Recognition Improvement of Mediapipe Model Based on Historical Trajectory Assist Tracking, Kalman Filtering and Smooth Filtering

Fan, Y (2024):

- Leverage filtering and historical trajectory tracking to improve detection through occlusion
- Best results came from added historical trajectory information and Kalman filtering
 - Required reduced resolution, leading to less accurate skeleton but overall better gesture detection
- Potential downside: increased computation, might lead to less real-time detection





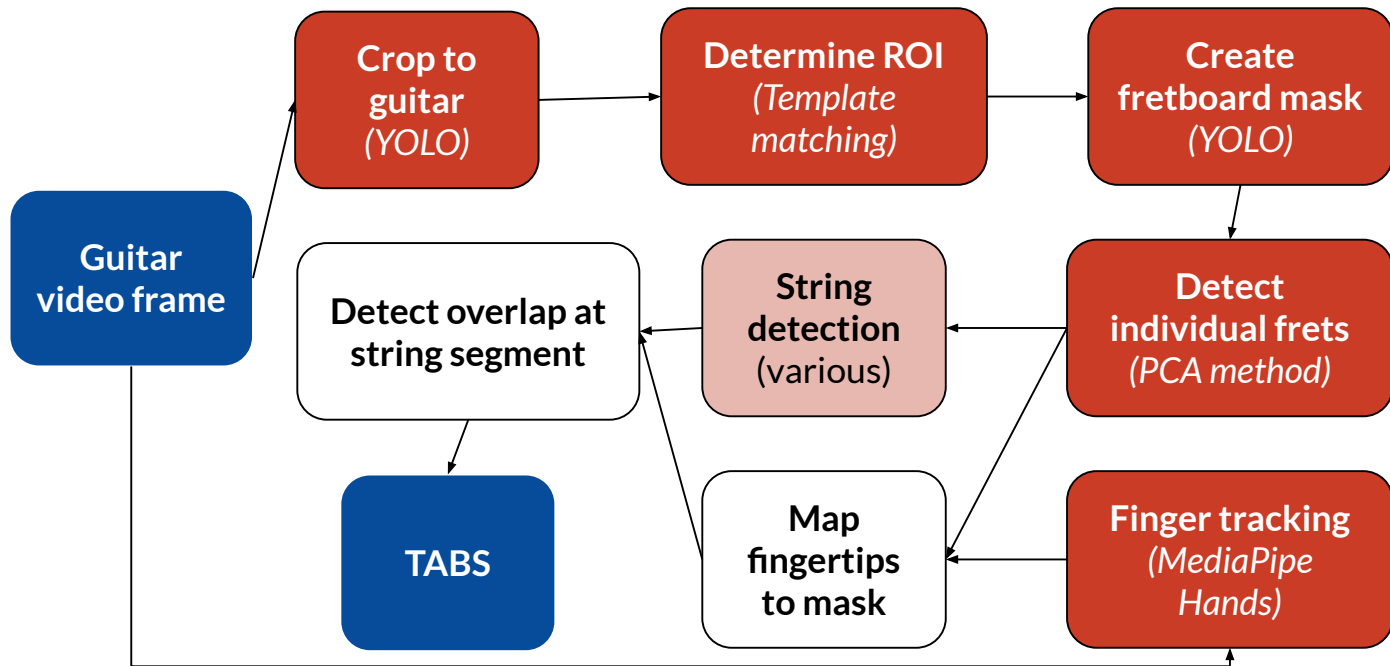
ZoeDepth: Zero-shot Transfer by Combining Relative and Metric Depth

- Improvement on MiDAS
- Makes inferences with metric units, as well as unitless estimations
- Combines both approaches for extra accuracy
- Previously maintained by Intel

Sep 8 2026 Update

Computer Vision Transcription

Arnav, Vishaal, Sooji



Bringing Fret, String and Fingertip detection together

- Objective: Transcribe guitar playing to tablature using Computer Vision only (no audio).
 - Best Fret Detection (Method H): EfficientNet-B0 regression achieves F1@2% of 0.835.
 - Best String Detection (Projection): Auto-Tune + Interpolation methodology achieves a 96% PASS rate.
 - Hand Tracking: MediaPipe HandLandmarker tracks 21-point hand joints.
-

Workflow

- Fretboard Segmentation: YOLOv8 extracts a precise binary mask of the guitar neck.
 - Canonical Warp: The mask's PCA corners are used to apply a perspective transform, stretching the fretboard into a flat 1024×256 "canonical space".
 - Fret Localization: Method H predicts 21 vertical lines in canonical space (with a mathematical PCA fallback).
 - String Localization: The Projection Auto-Tune method collapses the image horizontally to find exactly 6 horizontal valleys.
 - The Grid: The intersection of these lines creates a perfect 6×21 coordinate system for note mapping.
-

TAB Generation Methodology

Mapping Hands to the Grid

With the canonical coordinate grid established, note detection becomes a geometric problem.

- MediaPipe identifies fingertip (TIP) and second joint (PIP) coordinates in the raw video.
 - A finger is considered "pressing" the string only if the TIP is geometrically lower than the PIP joint.
 - The (x,y) pixel of a pressing fingertip is pushed through the perspective homography matrix into the canonical 1024×256 space.
 - The system finds the closest String Y line and the bounding Fret X zone.
 - To eliminate CV jitter, a note (String, Fret) must be registered in the majority of a 5-frame sliding window before it is emitted to the ASCII TAB overlay.
-

Sep 15 2026 Update

Computer Vision Transcription

Arnav, Vishaal, Sooji

String Detection review

Horizontal projection -> Peak detection -> Top 6 strings

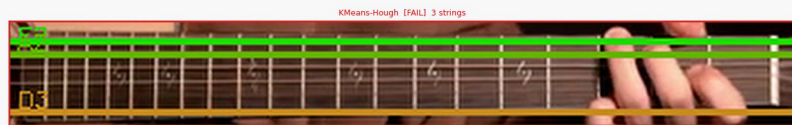
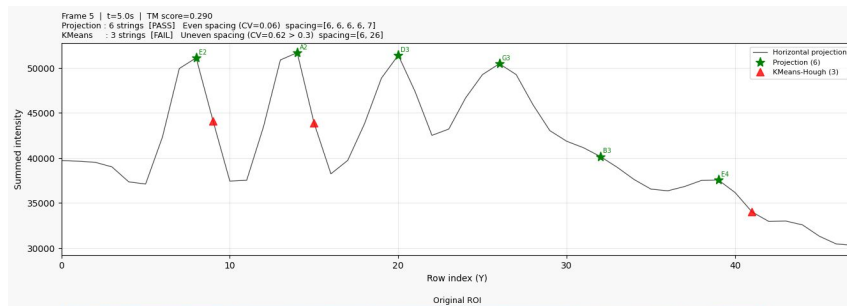
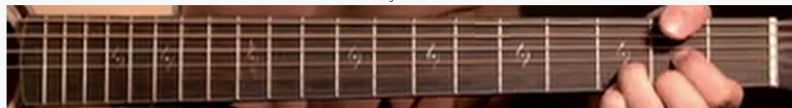
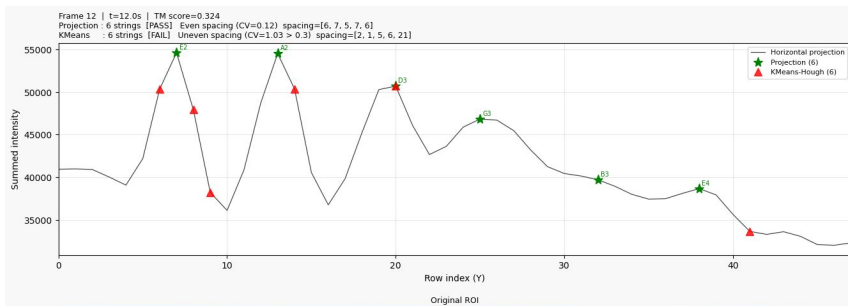
Key ideas

- **Projection:** Sum pixel brightness per row -> each string becomes a peak
- **Auto-Tune:** Rank peaks by prominence instead of a fixed threshold ($\text{mean} \times 0.9$)
- **Interpolation:** If only 5 strings are found, fill in the missing one using string spacing

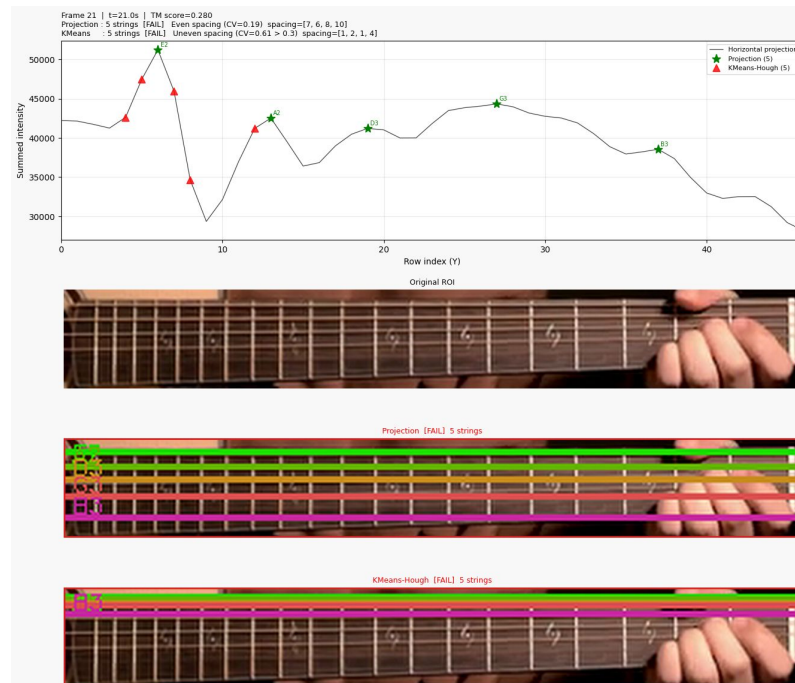
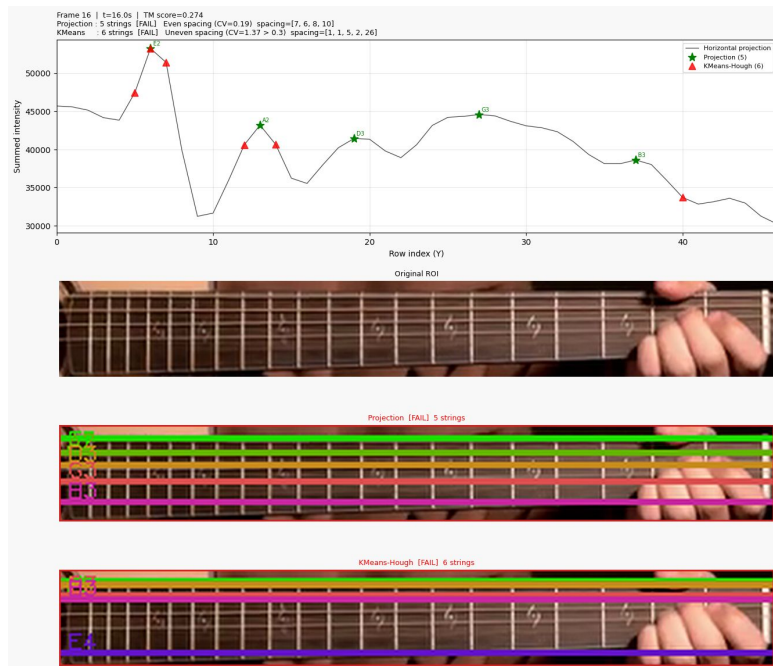
Result: PASS rate 74%(Asmar 2022) -> 96%

Limitations

- Assumes strings are horizontal -> fails on tilted ROI
 - Noise peaks can replace a missing string
 - PASS checks spacing only, not true position
-



Fail case



Update on String Detection

- **Tilted strings:** Model each string as $y = m \cdot x + b_i$ (shared slope). Pick the m that makes projection peaks sharpest.
 - **Noise peaks:** Drop low-prominence peaks, and search each string only near its expected position in each strip.
-

—

—

—

ViViT approach

- Video Vision Transformer (ViViT) for spatiotemporal modeling.
 - Demo uses pretrained ViViT model to detect fret events from video
 - Inputs: video frames -> canonical warp -> ViViT features
 - Outputs per frame: per-frame predictions and downstream TAB pipeline
-

ViViT pipeline

1. Preprocess crop and warp frames to canonical fretboard space
 2. ViViT backbone: extract spatiotemporal features
 3. Head per-frame regression classification (frets: strings)
 4. Postprocess: map predictions to image space and overlay
 5. Feed into TAB generator and visualization
-





Next steps

- Try generating tabs.
 - Try and solve string hover vs. press problem with a separate network.
 - Evaluate ViViT approach
 - Integrate string detection with ViViT.
-